Разработка Windows-приложений WPF

Практическое занятие 1. Создание приложений WPF 3

Упражнение 1. Создание приложения Windows *3*

Упражнение 2. Работа с окнами *8*

Упражнение 3. Создание навигационного приложения *11*

Практическое занятие 2. Применение макетов для организации элементов 12

Упражнение 1. Определение макета размещения элементов на странице *12*

Упражнение 2. Настойка интерфейса *14*

Практическое занятие 3. Маршрутизированные события 15

Упражнение 1. Создание обработчиков маршрутизированного события *15*

Упражнение 2. Использование маршрутизированных событий *16*

Упражнение 3. Применение маршрутизированных событий *17*

Практическое занятие 4. Работа с командами 18

Упражнение 1. Применение встроенных команд *18*

Упражнение 2. Создание и использование настраиваемой команды *19*

Практическое занятие 5. Создание целостного пользовательского интерфейса с применением стилей и шаблонов 21

Упражнение 1. Использование стилей для создания элементов *21*

Упражнение 2. Использование шаблонов для создания элементов *24*

Практическое занятие 6. Использование триггеров и анимации для создания интерактивного интерфейса 25

Упражнение 1. Использование триггеров для улучшения внешнего вида интерфейса *25*

Упражнение 2. Применение аанимационных эффектов в интерфейсе приложения *26*

Практическое занятие 7. Использование словарей ресурсов в приложении 29

Упражнение 1. Добавление готовых тем в приложение *29*

Упражнение 2. Создание и использование словаря ресурсов *31*

Практическое занятие 8. Привязка данных 33

Упражнение 1. Создание привязки данных к объекту-коллекции *33*

Практическое занятие 9. Использование компонента BackgroundWorker 36

Упражнение. Работа с компонентом BackgroundWorker *36*

Практическое занятие 10. Настройка интерфейса пользователя 38

Упражнение 1. Использование элементов Windows Forms *38*

Упражнение 2. Построение специального свойства зависимости *39*

Практическое занятие 11. Графика в приложениях WPF 43

Упражнение 1. Рисование замкнутой фигуры с помощью элемента "Многоугольник" *43*

Упражнение 2. Создание приложения рисования *44*
Упражнение 3. Создание диаграмм с использованием WPF Toolkit *45*
Практическое занятие 12. Использование параметров приложения 47
Упражнение 1. Работа с параметрами *47*
Лабораторная работа 1. Разработка приложения WPF 49
Упражнение 1. Создание инфраструктуры приложения *49*
Упражнение. 2 Реализация пользовательского интерфейса приложения *52*
Упражнение 3. Создание пользовательского интерфейса отдельной страницы *55*
Упражнение 4. Использование стилей *56*
Упражнение 5. Привязка данных к элементу управления *60*
Лабораторная работа 2. Разработка приложения на основе шаблона MVVM 64
Упражнение 1. Создание инфраструктуры приложения MVVM *64*
Упражнение 2. Реализация элементов шаблона *65*
Упражнение 3. Привязка команды кнопки к ViewModel *68*
Упражнение 4. Настройка обновления свойств *70*
Литература 72

Практическое занятие 1. Создание приложений WPF

В упражнениях этого занятия вы создадите простые приложения WPF с помощью конструктора и редактора XAML.

Упражнение 1. Создание приложения Windows

Изучение структуры проекта

1. Откройте Visual Studio и создайте новый проект – WPF Application, имя укажите WpfHello.

Проект откроется с окном по умолчанию с именем **MainWindow** в дизайнере и в режиме разметки кода XAML.

2. Перейдите в разметку XAML и выполните следующие действия:
   a. Проверьте, что в разметке элемент Window включен атрибут `x:Class`, в котором указывается класс окна, производный от класса Window. Этот класс объявлен в файле MainWindow.xaml.cs.
   b. Измените атрибут окна Title на текст по вашему усмотрению, установите значения высоты окна равным 350 (`Height="350"`), а ширины – 500 (`Width="500"`), также укажите минимально допустимое значение высоты (`MinHeight="350"`) и ширины (`MinWidth="300"`) окна.
   c. Обратите внимание на атрибуты `<Grid> </Grid>`, в которых должны размещаться элементы интерфейса – в окне должен быть только один элемент содержимого.
   d. Удалите атрибуты `<Grid> </Grid>` и замените их на элемент надписи:

```
<Label x:Name="lbl" HorizontalAlignment="Center"
VerticalAlignment="Top">Пуск</Label>
```

3. Проверьте, что второй элемент (например, тоже надпись) нельзя добавить в окно.
4. Откройте файл MainWindow.xaml.cs.
5. Проверьте, что в коде программной части класс окна объявлен классом `partial` (частичный) с тем же именем, которое задано атрибутом в разметке `x:Class` и является производным от класса Window. Класс реализует метод `InitializeComponent()`, который вызывается для регистрации событий и задания свойств, реализованных в разметке.
6. В конструкторе класса реализуйте присвоение содержимому элемента Label текста “Добрый день”, например:

```
lbl.Content = "Добрый день!";
```

7. Постройте и запустите приложение. Отобразится надпись в виде окна.

Размещение элементов управления

1. Добавьте в разметку окна теги компоновщика (они рассматриваются позже) `<Grid> </Grid>`
2. Перенесите элемент надпись `lbl` вовнутрь тегов `Grid`.
3. Постройте и запустите приложение. Функциональность не изменилась.

4. Перенесите из панели инструментов (**Toolbox)** на плоскость проекта две кнопки (**Button**), элементы **Textbox** и **Label,** разместите их так, как вам будет удобно или, например, как на рисунке 1.1.

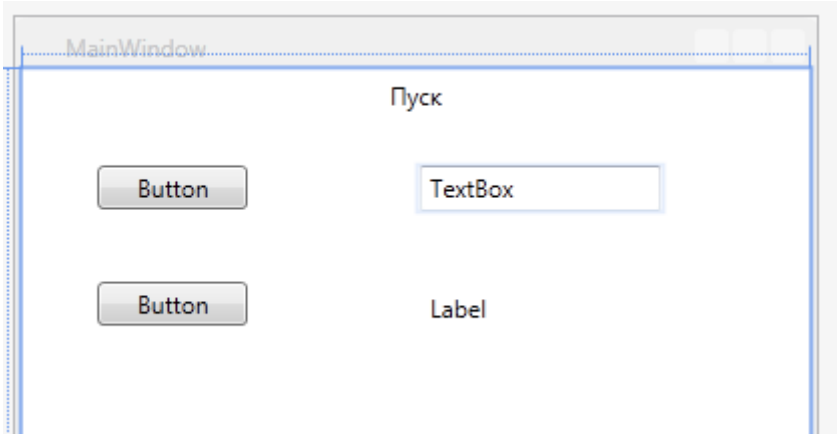


Рис. 1.1. Рабочая форма

5. Обратите внимание, что в теги `<Grid>` добавился код XAML размещения указанных элементов.
6. В окне **Properties** задайте для кнопок соответственно:
   a. свойству **Имя (Name)** значения setBut и retBut
   b. свойству **Content** значения: Set Name и Ret Name.
7. Для элемента **Label** установите:
   a. свойству **Имя (Name)** значение retLabel
   b. свойство **Content** сделайте пустым.
8. Для элемента **TextBox**:
   a. свойству **Имя (Name)** значение setText
   b. очистите свойство **Text**.
9. Перейдите для файла MainWindow.xaml в режим разметки XAML.
10. В разметке XAML файла MainWindow.xaml найдите элемент **Grid** и изучите его содержимое: там должны быть указаны настроенные вами свойства.

Замечание. Свойство **HorizontalAlignment** выравнивает элемент по горизонтали относительно правой или левой стороны контейнера (в данном случае окна) (Left, Right), ориентировать по центру (Center) ил растягивать по всей ширине (Stretch). Свойство **VerticalAlignment** задает для элемента выравнивание по вертикали: Top (положение в верху контейнера), Bottom (положение внизу), Center (положение по центру), Stretch (растяжение по всей высоте). Свойство **Margin** устанавливает отступы вокруг элемента: `Margin="левый_отступ, верхний_отступ, правый_отступ, нижний_отступ"`.

Добавление событий

1. Выделите первую кнопку `setBut`, в окне **Properties** перейдите на вкладку установки событий (с помощью значка «молния») и дважды кликните в поле **Click**, в добавленном обработчике события реализуйте возможность записи в текстовый файл

введенных данных в текстовое поле `setText`. Возможно решение будет следующим код (путь к файлу можете указать явно, например, `D:\\username.txt`):

```
private void setBut_Click(object sender, RoutedEventArgs e)
{
    System.IO.StreamWriter sw = null;
    try
    {
        sw = new System.IO.StreamWriter("username.txt");
        sw.WriteLine(setText.Text);
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
    finally
    {
        if (sw != null)
            sw.Close();
    }
}
```

2. Аналогичным образом добавьте обработчик события щелчка для второй кнопки, например:

```
private void retBut_Click(object sender, RoutedEventArgs e)
{
    System.IO.StreamReader sr = null;
    try
    {
      using (sr = new System.IO.StreamReader("username.txt"))
      retLabel.Content = "Приветствую Вас, уважаемый " + sr.ReadToEnd();
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
    finally
    {
        if (sr != null)
            sr.Close();
    }
```

3. Постройте и запустите приложение. Введите имя в текстовое поле и щелкните кнопку *Set Name*. Затем кнопку *Ret Name*, проверьте, что отобразилось имя в элементе **Label**.

В этой части упражнения вы реализуете сценарий, в котором после загрузки приложения кнопки недоступны для работы с ними, как только начинается ввод данных в текстовое

поле первая кнопка становится доступной, а после клика по ней становится активной и вторая кнопка.

1. Откройте файл MainWindow.xaml в режиме кода.
2. Добавьте в конструктор класса окна после вызова метода инициализации компонентов код установки свойства доступности (**IsEnabled**) кнопок в значение **false**:

```
public MainWindow()
{
    InitializeComponent();
    lbl.Content = "Добрый день!";
    setBut.IsEnabled = false;
    retBut.IsEnabled = false;
}
```

3. Перейдите в режим конструктора, с помощью окна свойств (раздел событий) добавьте для текстового поля событие, которое происходит при вводе данных – `TextChanged`.
4. В обработчик сгенерированного события добавьте код включения доступности первой кнопки:

```
private void setText_TextChanged(object sender, TextChangedEventArgs e)
{
    setBut.IsEnabled = true;
}
```

5. В обработчик клика первой кнопки после закрытия файлового потока добавьте код включения доступности второй кнопки:

```
    retBut.IsEnabled = true;
```

6. Постройте и запустите приложение. Проверьте, что кнопки после запуска неактивны. Введите имя в текстовое поле и проверьте, что кнопка *Set Name* стала активной. Щелкните по ней и проверьте доступность второй кнопки.

В этой части упражнения вы реализуете сценарий, в котором после закрытия окна приложения должно появиться небольшое оконное сообщение с вопросом о выборе из двух вариантов: либо продолжить закрытие окна без сохранения данных, либо отменить закрытие окна. Одна из распространенных причин, препятствующих закрытию окна, заключается в том, что содержимое окна содержит измененные данные. Событие **Closing** возникает перед закрытием окна и предоставляет механизм, с помощью которого можно предотвратить закрытие окна.

1. Перейдите в режим конструктора, с помощью окна свойств (раздел событий) добавьте для окна обработчик события, которое происходит перед закрытием окна – Closing.
2. В поле класса окна объявите переменную для контроля за изменениями данных:

```
bool isDataDirty = false;
```

3. Добавьте подтверждение сохранности данных в обработчике клика по кнопке setBut после закрытия объекта потока:

```
isDataDirty = false;
```

4. Добавьте подтверждение не сохранности данных в обработчике события изменения текста (`setText_TextChanged`) в поле setText:

```
isDataDirty = true;
```

5. В обработчике сгенерированного события Closing добавьте код контроля за сохранением данных:

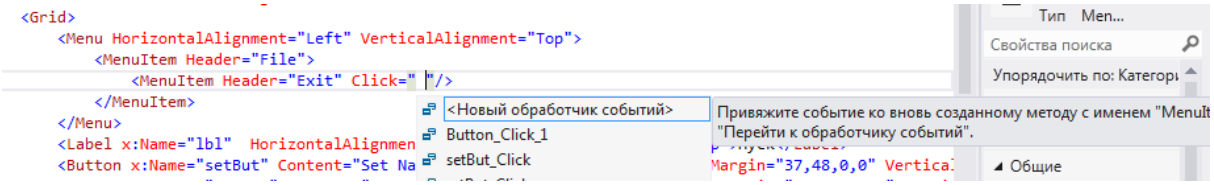
```
    private void Window_Closing_1(object sender,
System.ComponentModel.CancelEventArgs e)
    {
        if (this.isDataDirty)
        {
            string msg = "Данные были изменены, но не сохранены!\n Закрыть
окно без сохранения?";
            MessageBoxResult result = MessageBox.Show(msg, "Контроль
данных", MessageBoxButton.YesNo, MessageBoxImage.Warning);
            if (result == MessageBoxResult.No)
            {
                e.Cancel = true;
            }
        }
    }
```

6. Постройте и запустите приложение. Проверьте работу добавленного функционала.

Использование редактора XAML

В этой части упражнении вы научитесь добавлять элементы управления не только с помощью панели Toolbox, но и с помощью кода XAML в редакторе WPF приложения.

1. Перейдите в режиме разметки XAML файла MainWindow.xaml.
2. Поставьте курсор после тега `<Grid>` и нажмите клавишу **Enter**.
3. С новой строки начните вводить текст `<Menu Horizo…` , используйте подсказки редактора, введите код XAML, показанный на рисунке (после ввода Click появится подсказка «Новый обработчик событий») – кликните по нему дважды):



В результате код меню должен быть таким (а в окне дизайнера должно появиться меню):

```
<Menu HorizontalAlignment="Left" VerticalAlignment="Top">
    <MenuItem Header="File">
        <MenuItem Header="Exit" Click="MenuItem_Click_1"/>
    </MenuItem>
</Menu>
```

4. Добавьте код в тело обработчика пункта Exit для закрытия текущего окна:

```
    private void MenuItem_Click_1(object sender, RoutedEventArgs e)
    {
```

```
        this.Close();
    }
```

5. Постройте и протестируйте функциональность меню.

Упражнение 2. Работа с окнами

В этом упражнении вы добавите в проект второе окно и реализуете взаимодействие между ним и главным окном.

Добавление нового окна

1. С помощью меню **Project (Проект)** добавьте новое окно с именем MyWindow.
2. Откройте файл App.xaml.
3. В теге приложения найдите атрибут `StartupUri="MainWindow.xaml",` указывающий загружаемое окно.
4. Укажите файл MyWindow.xaml для загрузки нового окна в качестве стартового.
5. Запустите приложение, проверьте, что открылось новое окно.
6. Верните значение атрибута `StartupUri` в `MainWindow.xaml.`
7. Добавьте еще одну кнопку в окно MainWindow.
8. Задайте для кнопки свойство **Name** значение New_Win, а свойству **Content** значение New Win.
9. Справа от новой кнопки добавьте элемент – текстовый блок **TextBlock** с именем `x:Name="txtBlock"` и следующими атрибутами:
   a. поддержкой переноса текста `TextWrapping="Wrap"`
   b. выравниванием `VerticalAlignment="Top" HorizontalAlignment="Left"`
   c. размерами `Height="50" Width="75"`
10. Добавьте обработчик события **Click** для новой кнопки.
11. Чтобы открывать второе окно в одном экземпляре (недопустимо, чтобы при повторных кликах по кнопке создавался каждый раз новый экземпляр окна) выполните следующее:
    a. в поле класса <u>главного</u> окна объявите ссылку на новое окно:
    ```
    public MyWindow myWin { get; set; }
    ```
    b. в обработчике события **Click** сначала проверьте на **null** ссылку на экземпляр нового окна и в случае **true** создайте новый экземпляр, а если экземпляр уже существует, то вызовите метод `Show()`:
    ```
    if (myWin == null)
        myWin = new MyWindow();
    myWin.Show();
    ```
    c. в поле класса <u>второго</u> окна объявите логическую переменную для контроля за закрытием окна:
    ```
    private bool _close;
    ```

d. определите новый метод закрытия окна (он будет скрывать существующий метод в базовом классе):

```
public new void Close()
{
    _close = true;
    base.Close();
}
```

e. добавьте для нового окна обработчик события Closing, в котором должно проверяться переменная _close и если будет разрешено закрыть окно, то вместо закрытия его следует скрыть:

```
private void Window_Closing_1(object sender,
System.ComponentModel.CancelEventArgs e)
{
    if (_close) return;
    e.Cancel = true;
    Hide();
}
```

12. Постройте и запустите приложение. Проверьте, что по клику третьей кнопки открывается вторая форма.
13. Закройте главное окно – обратите внимание, что второе окно не закрылось, его приходится закрывать индивидуально.

Замечание. Для некоторых окон требуется связь с окном, которое их открывает. Это необходимо в случае, когда окна должны всегда закрываться, сворачиваться, разворачиваться и восстанавливаться вместе с окном, которое их создало или требуется передать данные из второго окна в главное (связанное с ним) окно. Такая связь может быть реализована путем создания одного окна другим и установки свойства Owner принадлежащего окна со ссылкой на окно владельца.

14. В обработчике события **Click** кнопки создания окна после создания экземпляра нового окна присвойте свойству Owner ссылку на текущее (главное) окно:

```
if (myWin == null)
    MyWindow myWin = new MyWindow();
myWin.Owner = this;
myWin.Show();
```

15. Постройте и запустите приложение. Проверьте, что по клику третьей кнопки открывается второе окно, а после закрытия главного окна оно тоже закрылось.

Установка расположения окон

В этой части упражнения вы укажите расположение окон в координатах x и y относительно рабочего стола. Это расположение можно определить с помощью свойств **Left** и **Top**.

1. Добавьте в конец конструктора класса окна MainWindow настройку свойств расположения главного окна:

```
Top = 25;
Left = 25;
```

2. В обработчике события **Click** кнопки создания окна перед вызовом метода Show() настройте расположение нового окна – оно должно открываться справа от главного окна:

```
myWin.Top = this.Top;
myWin.Left = this.Left + this.Width;
```

3. Постройте и запустите приложение. Проверьте, что по клику третьей кнопки открывается второе окно справа от главного окна, а после изменения ширины главного окна повторный клик по кнопке приводит к перемещению нового окна в установленное место.

Замечание. Представляет интерес местоположение открытия нового окна «привязать» к кнопке, которая ее вызывает. Например, для того чтобы окно открывалось справа от кнопки, необходимо рассчитать ее экранные координаты и затем использовать их при определении места открытия нового окна.

4. Для реализации указанной идеи измените строки настройки местоположения окна на следующие:

```
var location = New_Win.PointToScreen(new Point(0, 0));
myWin.Left = location.X + New_Win.Width;
myWin.Top = location.Y;
```

5. Постройте и запустите приложение. Проверьте, что по клику третьей кнопки открывается второе окно справа от нее, а после изменения ширины главного окна или смещения повторный клик по кнопке приводит к перемещению нового окна в установленное место – рядом с кнопкой.

Передача данных из нового окна в главное окно

1. Перейдите в окно дизайнера второго окна и перенесите на поверхность окна кнопку и текстовое поле.
2. Для текстового поля укажите имя (`x:Name="textBox"`), возможность переноса текста но новою строку (`TextWrapping="Wrap"`), высоту (`Height="80"`) и ширину (`Width="120"`).,
3. В поле класса окна добавьте ссылку на главное окно:

```
MainWindow wnd1 = null;
```

4. Добавьте обработчик события **Click** для кнопки и в обработчике:
   a. инициализируйте ссылку `wnd1` с помощью свойства Owner,
   b. присвойте свойству Text текстового блока txtBlock главного окна текст текстового поля второго окна,
   c. и, если требуется после передачи данных закрытие второго окна – вызовите для него метод Close():

```
private void Button_Click(object sender, RoutedEventArgs e)
{
    wnd1 = Owner as MainWindow;
    if (wnd1 != null)
    {
        wnd1.txtBlock.Text = textBox.Text;
```

```
        }
        Close();
    }
```

Замечание. Из другого окна можно обратиться только к тем элементам главного окна, у которых указан атрибут **Name**.

5. Постройте и запустите приложение. Откройте второе окно. Введите текст в текстовое поле и проверьте, что по клику кнопки второго окна данные отобразились в главном окне, а второе окно закрылось.
6. Запустите приложение в режиме отладки с точной останова на проверке `if (myWin == null)` в обработчике открытия второго окна. Проверьте, как работает контроль при первом открытии второго окна. Откройте второе окно после его закрытия и проверьте значение ссылки myWin – должно быть не null. Продолжение выполнения приложения должно привести к появлению исключительной ситуации.
7. Для решения возникшей проблемы следует при удалении окна обнулить ссылку на него. Добавьте обработчик события, возникающее после закрытия окна – `Closed` и в нем обнулите ссылку:

```
private void Window_Closed(object sender, EventArgs e)
{
    wnd1.myWin = null;
}
```

8. Повторите процесс отладки и проверьте, что при закрытии окна его можно повторно создать и открыть.

Логирование

1. При необходимости можно записывать данные в файл журнала. Для реализации этой функциональности добавьте в класс второго окна следующий метод:

```
 using (System.IO.StreamWriter writer = new System.IO.StreamWriter("log.txt",
true))
 {
   writer.WriteLine("Внесено {0}: {1} ", textBox.Text,
DateTime.Now.ToShortDateString() + ", время: " +
        DateTime.Now.ToLongTimeString());
   writer.Flush();
 }
```

2. В обработчике события клика по кнопке вызовите этот метод:

```
wnd1.txtBlock.Text = 11extbox.Text;
PrintLogFile();
```

3. Постройте и запустите приложение. Проверьте функционирование записи данных в файл журнала.

Упражнение 3. Создание навигационного приложения

В этом упражнении вы создадите приложение навигационного типа (NavigationWindow).

1. Откройте Visual Studio и создайте новый проект – WPF Application, имя укажите WpfHelloNav.

2. Откройте MainWindow.xaml в режиме разметки XAML и замените тег `Window` на `NavigationWindow`.
3. Удалите тэг `<Grid></Grid>`, так как `NavigationWindow` его не поддерживает.
4. Откройте файл MainWindow.xaml в режиме кода и замените базовый класс `Windows` заменяем на `NavigationWindow`.
5. Постройте и запустите приложение. Окно будет со стрелками навигации.
6. Добавьте в приложение две страницы WPF. Для этого в меню **Project** выберите **Add…(Добавить) Page (Страницу),** оставьте имена страниц без изменений.
7. Откройте файл App.xaml в редакторе XAML.
8. С помощью конструктора для главного окна MainWindow установите свойство `Source` значением Page1.xaml, в коде разметки появится соответствующая запись:

```
Title="MainWindow" Height="350" Width="525" Source="Page1.xaml">
```

9. Откройте первую страницу в режиме дизайна, на поверхность страницы перенесите кнопку.
10. Добавьте для кнопки первой страницы обработчик события клика и укажите в нем код создания экземпляра второй страницы:

```
private void button_Click(object sender, RoutedEventArgs e)
{
    Page2 page = new Page2();
    this.NavigationService.Navigate(page);
}
```

11. Постройте и запустите приложение. Кликните по кнопке, произойдет переход на вторую страницу. Проверьте работу навигационных кнопок.

Данный тип приложения реализуется в лабораторной работе 1.

Практическое занятие 2. Применение макетов для организации элементов

В упражнениях этого занятия вы для организации размещения и упорядочения элементов в главном окне используете компоненты, управляющие отрисовкой элементов (их размер и расположение, а также размеры и расположение вложенных в них элементов).

Существует шесть классов-наследников от класса Panel (Canvas, DockPanel, Grid, StackPanel, VirtualizingStackPanelи WrapPanel), разработанные специально для создания интерфейсов приложений. В этом задании будут использоваться компоненты Grid и DockPanel.

Упражнение 1. Определение макета размещения элементов на странице

В этом упражнении в качестве внешнего контейнера будет использоваться элемент **DockPanel**, а кнопки и связанные с ними текстовые элементы будут размещены в сетке – элементе **Grid**.

Замечание. Элемент **DockPanel** используется для размещения содержимого дочерних элементов вдоль края контейнера макета с помощью свойства **DockPanel.Dock** этих дочерних элементов. Элемент **Grid** используется для точного размещения содержимого в строках и столбцах.

1. Откройте приложение, созданное на первом занятии, и перейдите в режим разметки XAML главного окна `MainWindow.xaml`
2. Перед тегом `<Grid>` вставьте пару тегов `<DockPanel>`:

```
<DockPanel>

</DockPanel>
```

3. Перенесите меню в элемент **DockPanel**, укажите атрибут `DockPanel.Dock="Top"`:

```
<DockPanel>
    <Menu DockPanel.Dock="Top" HorizontalAlignment="Left">
        <MenuItem Header="File">
            <MenuItem Header="Exit" Click="MenuItem_Click_1"/>
        </MenuItem>
    </Menu>
</DockPanel>
```

4. Разместите элемент Label (`x:Name="lbl"`) после закрывающегося тега меню и установите атрибут выравнивания `HorizontalAlignment="Center"` и атрибут стыковки `DockPanel.Dock ="Top"`:

```
    </Menu>
<Label x:Name="lbl"  HorizontalAlignment="Center" DockPanel.Dock ="Top">Пуск</Label>
```

Замечание. Часто в приложениях используется строка состояния (статусная панель), располагающаяся, как правило, внизу окна. Класс StatusBar предоставляет элемент – статусную панель, отображающую элементы и информацию на горизонтальной полосе в окне приложения.

5. После элемента Label (`x:Name="lbl"`) добавьте следующий XAML-код, определяющий статусную панель, содержащую три элемента:

```
<StatusBar x:Name="stb" DockPanel.Dock="Bottom">
    <StatusBarItem x:Name="stb1">
        <TextBlock>Статусная панель</TextBlock>
    </StatusBarItem >
    <Separator/>
    <StatusBarItem x:Name="stb2">
        <TextBlock>Online</TextBlock>
    </StatusBarItem>
    <StatusBarItem x:Name="stb3">
        <TextBlock></TextBlock>
    </StatusBarItem>
</StatusBar>
```

6. После статусной панели добавьте теги `<Grid>`, с помощью элементов ColumnDefinition и RowDefinition определите два столбца и три строки:

```
<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
        <RowDefinition Height="*" />
        <RowDefinition Height="*" />
        <RowDefinition Height="*" />
    </Grid.RowDefinitions>
</Grid>
```

7. Перед закрывающимся тегом `</Grid>` перенесите кнопки и текстовые элементы (`x:Name="retLabel"`, `x:Name="txtBlock"`) с указанием с помощью присоединенных свойств (`Grid.Column` и `Grid.Row`) расположение элемента в определенной ячейке сетки:

```
        <DockPanel>
            <Menu DockPanel.Dock="Top" HorizontalAlignment="Left">
              ...
            </Menu>
        <Label x:Name="lbl"  HorizontalAlignment="Center" DockPanel.Dock ="Top">Пуск</Label>
        <StatusBar x:Name="stb" DockPanel.Dock="Bottom">
              ...
        </StatusBar>
        <Grid>
            <Grid.ColumnDefinitions>
            ...
            </Grid.RowDefinitions>

            <Button x:Name="setBut" Grid.Column="0" Grid.Row="0" Content="Set Name "
HorizontalAlignment="Left" Margin="10,15" VerticalAlignment="Top" Width="75"
Click="setBut_Click"/>
            <Button x:Name ="retBut" Grid.Column="0" Grid.Row="1" Content="Ret Name "
HorizontalAlignment="Left" Margin="10,15" VerticalAlignment="Top" Width="75"
Click="retBut_Click"/>
            <TextBox x:Name="setText" Grid.Column="1" Grid.Row="0"
HorizontalAlignment="Left" VerticalAlignment="Top"  Height="23" Width="120" Margin="10,15"
TextChanged="setText_TextChanged"/>
            <Label x:Name="retLabel" Grid.Column="1" Grid.Row="1" Content="Label"
HorizontalAlignment="Left" Margin="10,15" VerticalAlignment="Top"/>
            <Button x:Name="New_Win" Grid.Column="0" Grid.Row="2"  Content="New Win"
HorizontalAlignment="Left" Margin="10,15" VerticalAlignment="Top" Width="75"
Click="New_Win_Click_1"/>
            <TextBlock x:Name="txtBlock" Grid.Column="1" Grid.Row="2" Margin="10,15"
VerticalAlignment="Top" TextWrapping="Wrap"/>
        </Grid>

    </DockPanel>
```

8. Постройте и запустите приложение. Проверьте, что при такой настройке ширины столбцов в случае расширения окна приложения столбцы растягиваются, а текстовые поле и блок статичны, что является неестественным с точки зрения ожидания поведения интерфейса.

Упражнение 2. Настойка интерфейса

В этом упражнении вы внесете изменения в настройку некоторых элементов так, чтобы их поведение в процессе изменения размеров окна было бы естественным и ожидаемым. Например, при расширении окна текстовые элементы должны тоже растягиваться.

1. Задайте ширину первого столбца точное значение, например, 150. В этом случае растягиваться будет только второй столбец:

```
        <Grid.ColumnDefinitions>
                <ColumnDefinition Width="150"/>
```

2. Задайте для текстового поля (`TextBox x:Name="setText"`) горизонтальное выравнивание «растяжение» (`HorizontalAlignment="Stretch"`) и удалите настройку ширины этого элемента.
3. Для текстового блока (`TextBlock x:Name="txtBlock"`) удалите атрибуты горизонтального и вертикального выравнивания, а также атрибуты размера (~~`VerticalAlignment="Top"  HorizontalAlignment="Left"  Height="50"  Width="75"`~~), а атрибут, задающий точные отступы от краев контейнера укажите в полном виде: `Margin="10,15,10,15".`

Указание. Текстовое поле можно было бы изменить также как и текстовый блок, и наоборот. Также можно задать возможный диапазон ширины и высоты с помощью свойств MinWidth/MaxWidth и MinHeight/MaxHeight. И при растяжении или сжатии контейнеров элементы с данными заданными свойствами не будут выходить за пределы установленных значений.

4. Постойте и запустите приложение. Сравните поведение элементов при изменении ширины окна в процессе работы приложения.

Практическое занятие 3. Маршрутизированные события

В предлагаемом упражнении вы на основе маршрутизированных событий создадите обработчики событий в трех различных элементах управления в визуальном дереве, а также проанализируете порождение и обработку события каждым элементом управления.

Упражнение 1. Создание обработчиков маршрутизированного события

1. Создайте новый проект WPF Application по имени WpfRoutedEvent_1.
2. Перетащите из **Toolbox** на плоскость проекта элемент **Textbox** и три элемента **RadioButton** разместите их так, как вам будет удобно.
3. Содержимое элемента **Textbox** сделайте пустым (очистите свойство Text).
4. Задайте отображение содержимого элементов управления **RadioButton** либо непосредственно в представлении XAML, либо в окне свойств (**Content**):

radioButton1 – Обработка события в Textbox

radioButton2 – Обработка события в Grid

radioButton3 – Обработка события в Window

5. С помощью окна свойств присвойте имена этим элементам – radBut1, radBut2 и radBut3 соответственно.
6. С помощью окна свойств (вкладка событий) или в представлении XAML добавьте для элемента **TextBox** обработчик события `TextChanged`. Проверьте, что в коде класса окна создан соответствующий обработчик события.
7. В представлении XAML элемента **Grid** введите `TextBoxBase.TextChanged` и нажмите дважды клавишу =, в сплывающей подсказке выберите **Новый обработчик событий**, добавится соответствующее значение параметра. Запись обработчика должна выглядеть следующим образом:

```
<Grid TextBoxBase.TextChanged="Grid_TextChanged_1">
```

8. Повторите предыдущий пункт для элемента Window. Запись обработчика должна выглядеть следующим образом:

```
<Window TextBoxBase.TextChanged="Window_TextChanged_1"  x:Class=...
```

9. В коде добавьте в соответствующие обработчики событий, следующие фрагменты кода:

Для `textBox1_TextChanged_1`:

```
MessageBox.Show("Event by textbox");
```

```
e.Handled = (bool)radBut1.IsChecked;
```

Для `Grid_TextChanged_1`:

```
MessageBox.Show("Event by Grid");
e.Handled = (bool)radBut2.IsChecked;
```

Для `Window_TextChanged`:

```
MessageBox.Show("Event by Window");
e.Handled =(bool)radBut3.IsChecked;
```

10. Постройте и запустите приложение.
11. В поле **TextBox** введите букву. Отобразятся по очереди три сообщения, каждое из которых указывает элемент управления, породивший событие. Проверьте, что, выбирая переключатели можно остановить порождение пузырькового события в обработчиках.

Замечание. Для остановки дальнейшего инициирования событий свойству `Handled` присваивается значение true.

Упражнение 2. Использование маршрутизированных событий

В этом упражнении вы используете механизм маршрутизируемых событий для реализации однотипных действий.

1. Создайте новый проект – WPF Application по имени WpfRoutedEvent_2.
2. Перетащите из **Toolbox** на плоскость проекта элементы **Textbox** и **StackPanel**, в который поместите две кнопки **Button**.
3. Для элементов **Textbox** и кнопок укажите имена (свойство Name) `txtBox`, `butAdd` и `butMult` соответственно.
4. Откройте код XAML и для элемента добавьте событие `ButtonBase.Click`:

```
<StackPanel ButtonBase.Click="StackPanel_Click_1"  HorizontalAlignment="Left"
Margin="350,40,0,0" VerticalAlignment="Top" Width="100">
    <Button x:Name="butAdd" Content="x+x"/>
    <Button x:Name="butMult" Content="x*x"/>
</StackPanel>
```

5. Перейдите в режим кода и в сгенерированный обработчик элемента **StackPanel** добавьте код, реализующий считывание числа из текстового поля в переменную и выбор действия с этим числом – первая кнопка складывает, вторая – умножает число на саму себя:

```
private void StackPanel_Click_1(object sender, RoutedEventArgs e)
{
    FrameworkElement feSource = e.Source as FrameworkElement;
    double a = Double.Parse(txtBox.Text);

    switch (feSource.Name)
    {
        case "butAdd":
            a += a;
            break;
```

```
            case "butMult":
                a *= a;
                break;
        }
        e.Handled = true;

        txtBox.Text = String.Format("{0:#.##}", a);
    }
```

6. Постройте и запустите приложение. Введите в текстовое поле число и проверьте работу кнопок.
7. Добавьте в панель третью кнопку для извлечения квадратного корня и аналогично вышеописанному способу реализуйте функциональность добавленной кнопки.

Упражнение 3. Применение маршрутизированных событий

В этом упражнении вы измените обработку событий в приложении, разработанном на первых двух занятиях. Для двух обработчиков событий клика по кнопкам setBut и retBut вы реализуете один обработчик на уровне сетки.

1. Откройте приложение, созданное по итогам второго занятия.
2. В поле класса главного окна объявите переменную строкового типа и проинициализируйте ее именем файла для сохранения данных:

```
string nameFile = "username.txt";
```

3. На основе логики обработчиков событий кнопок setBut и retBut реализуйте соответствующие методы, например, для кнопки setBut:

```
private void SetBut()
{
    System.IO.StreamWriter sw = new System.IO.StreamWriter(nameFile);
    sw.WriteLine(setText.Text);
    sw.Close();
    retBut.IsEnabled = true;
    isDataDirty = false;
}
```

4. Реализуйте аналогичным способом метод `Retbut()` для кнопки retBut.
5. Для сетки `Grid` добавьте в коде XAML обработчик события клика по кнопке:

```
<Grid Button.Click="Grid_Click">
    <Grid.ColumnDefinitions>
        …
```

6. В коде главного окна реализуйте созданный обработчик события по аналогии с предыдущим упражнением:

```
private void Grid_Click(object sender, RoutedEventArgs e)
{
    FrameworkElement feSource = e.Source as FrameworkElement;
    try
    {
```

```
            switch (feSource.Name)
            {
            case "setBut":
                SetBut();
                break;
            case "retBut":
                RetBut();
                break;
            }
            e.Handled = true;
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message);
        }
    }
```

7. Удалите реализацию обработчиков событий клика по кнопкам setBut и retBut, а также удалите соответствующие атрибуты в XAML-коде этих кнопок (`Click="…Click"`).
8. Постройте и запустите приложение. Функциональность должна остаться прежней.

Практическое занятие 4. Работа с командами

В упражнениях этого занятия вы реализуете взаимодействие с пользователем с помощью команд.

Замечание. В WPF кроме обработки событий приложение может содержать команды, представляющие собой механизм выполнения какой-нибудь задачи, в процессе выполнения которой, все действия объединяются в один объект – команду. Использование команд помогает сократить объем кода и использовать одну и ту же команду для нескольких элементов управления в различных местах программы. Таким образом, команды позволяют абстрагировать набор действий от конкретных событий конкретных элементов.

Упражнение 1. Применение встроенных команд

Замечание. WPF обладает большим набором встроенных команд. Все они представляют объекты класса **RoutedUICommand**, который является производным от **RoutedCommand**. Все встроенные команды объединяются в группы, и каждую такую группу представляет отдельный статический класс, а отдельные команды реализованы как статические свойства этих классов.

В этом упражнении вы используете функциональность группы общих команд приложения, представленные классом **ApplicationCommands**, работающих с буфером обмена: Copy (Копировать), Cut (Вырезать) и Paste (Вставить).

1. Откройте итоговый проект прошлого занятия.

2. В представлении XAML главного окна добавьте в меню следующий код для создания второй группы меню **Edit(Правка)**, содержащую пункты для работы с буфером обмена и настройки соответствующих встроенных команд:

```
<MenuItem Header="Edit">
  <MenuItem Header="Копировать" CommandTarget="{Binding ElementName=MainWindow}"
Command="Copy"/>
  <MenuItem Header="Вырезать" CommandTarget="{Binding ElementName=MainWindow}"
Command="Cut"/>
  <MenuItem Header="Вставить" CommandTarget="{Binding ElementName=MainWindow}"
Command="Paste"/>
</MenuItem>
```

Замечание. С помощью выражения `CommandTarget="{Binding ElementName=MainWindow}"` указывается, что команды будут направлены на элементы главного окна, которые будут целью команд.

3. Постройте и запустите приложение.
4. Введите текст в текстовое поле, выделите введенный текст и выберите команду **Копировать** меню **Edit (Правка)**. Откройте второе окно и вставьте текст из буфера обмена командой **Вставить** контекстного меню.
5. Повторите описанные действия с целью проверки функционирования клавиш быстрого доступа.
6. Откройте стороннее приложение, например, Блокнот и вставьте в него скопированный текст из буфера обмена.

Упражнение 2. Создание и использование настраиваемой команды

Замечание. Наиболее простой способ создания команды состоит в использовании классов **RoutedCommand** и **RoutedUICommand**, в которых реализован интерфейс **ICommand**. Класс **RoutedUICommand** добавляет свойство Text, в котором указывается текст, отображаемый вместе с данной командой, если же текст команды никогда не будет отображаться где-либо в пользовательском интерфейсе, то подойдет и класс RoutedCommand.

В конструктор этого объекта в данном случае передается название команды и элемент-владелец команды (здесь объект MainWindow).

В этом упражнении будет создана настраиваемая команда – объект класса **RoutedUICommand**, а затем подключена к элементам интерфейса с помощью *CommandBinding*.

1. Откройте проект предыдущего упражнения.
2. Добавьте в проект новый класс: в меню **Project** выберите команду **Add Class**, имя классу укажите – CustomCommands.
3. Добавьте в новый класс инструкцию:

```
using System.Windows.Input;
```

4. Команду определите как статическое свойство *только для чтения* с именем `Launch`, возвращающую экземпляр `RoutedUICommand`:

```
public static RoutedUICommand Launch { get; }
```

5. Добавьте в класс статический конструктор, который

a. создает новую коллекцию `InputGestureCollection`,

b. добавляет соответствующую клавишу ввода, связанную с новой командой (Ctrl+L)

c. и инициирует переменную, возвращающую настраиваемую команду:

```
static CustomCommands()
{
    InputGestureCollection myInputGestures = new
InputGestureCollection();
    myInputGestures.Add(new KeyGesture(Key.L, ModifierKeys.Control));
    Launch = new RoutedUICommand("Запуск", "Launch",
typeof(CustomCommands), myInputGestures);
}
```

6. В окне редактора XAML главного окна в раздел меню добавьте новый элемент – пункт меню:

```
<MenuItem Header="File">
    <MenuItem Header="Launch"/>
    <MenuItem Header="Exit" Click="MenuItem_Click_1"/>
</MenuItem>
```

7. Добавьте в окне редактора XAML главного окна новую элемент статусной панели – флажок **CheckBox**. Этот элемент, как предполагается, будет запрещать или разрешать выполнение команды:

```
<StatusBarItem x:Name="stb4">
    <CheckBox x:Name="check">Разрешить открытие формы</CheckBox>
</StatusBarItem>
```

8. Постройте приложение и проверьте функциональность интерфейса.

9. В представлении XAML добавьте следующий код (если его нет) в разметку окна, чтобы создать ссылку на класс, содержащий настраиваемую команду:

```
xmlns:Local="clr-namespace:имя_пространства_имен_вашего_проекта"
```

10. В представлении XAML добавьте атрибут `Command` с указанием команды в кнопку открытия нового окна:

```
<Button x:Name="New_Win"  Command="Local:CustomCommands.Launch"
```

и элемент меню **Launch**:

```
<MenuItem Header="Launch" Command="Local:CustomCommands.Launch"/>
```

Замечание. Чтобы связать эту команду с реальным кодом, который бы выполнял некоторые действия, нужно использовать привязку команд. Привязка команд представляет объект **CommandBinding**. Его событие `Executed` прикрепляет обработчик, который будет выполняться при вызове команды, а свойство `Command` устанавливает саму команду, к которой относится обработчик.

11. В конструктор класса **MainWindow** добавьте (после вызова метода инициализации компонентов или просто в конец конструктора) код, который создает и регистрирует привязку **CommandBinding** для команды **Launch**.

Эта привязка должна связать событие `Launch.Executed` с методом `Launch_Handler`, а событие `Launch.CanExecuted` – с методом `LaunchEnabled_Handler`:

```
            CommandBinding abinding = new CommandBinding();
            abinding.Command = CustomCommands.Launch;
abinding.Executed += new ExecutedRoutedEventHandler(Launch_Handler);
abinding.CanExecute += new CanExecuteRoutedEventHandler(LaunchEnabled_Handler);
            this.CommandBindings.Add(abinding);
```

12. В класс главного окна добавьте обработчик события `LaunchEnabled_Handler`, в котором будет разрешаться или запрещаться выполнение команды:

```
        private void LaunchEnabled_Handler(object sender,
 CanExecuteRoutedEventArgs e)
        {
            e.CanExecute = (bool)check.IsChecked;
        }
```

13. В класс главного окна добавьте обработчик события `Launch_Handler`, в который скопируйте весь код обработчика события клика по кнопке открытия нового окна (`New_Win_Click_1`):

```
        private void Launch_Handler(object sender, ExecutedRoutedEventArgs e)
        {
            if (myWin == null)
                myWin = new MyWindow();
            myWin.Owner = this;
            var location = New_Win.PointToScreen(new Point(0, 0));
            myWin.Left = location.X + New_Win.Width;
            myWin.Top = location.Y;
            myWin.Show();
        }
```

14. Удалите реализацию обработчика события клика по кнопке New_Win, а также удалите соответствующий атрибут в XAML-коде этой кнопки (`Click="New_Win_Click_1"`).
15. Постройте и запустите приложение. При запуске кнопка открытия нового окна и элемент меню отключены. Для включения команды требуется установить флажок. Включите флажок и проверьте функциональность приложения.

Практическое занятие 5. Создание целостного пользовательского интерфейса с применением стилей и шаблонов

Упражнение 1. Использование стилей для создания элементов

В данном упражнении будет переработан пользовательский интерфейс с целью придания элементам управления привлекательного вида с использованием стилей.

Замечание. Style предоставляет удобный способ применения набора значений свойств (например, шрифт, цвет фона, размер, заливка) к нескольким элементам.

1. Откройте итоговый проект занятия №4.

2. В коде XAML главного окна для кнопки **setBut** добавьте новые свойства (выделенные жирным шрифтом), а свойство – ширину элемента удалите:

```
<Button x:Name="setBut" Grid.Column="0" Grid.Row="0" Content="Set Name "
HorizontalAlignment="Left" Margin="10,15"  VerticalAlignment="Top" Foreground ="Gold"
Background = "Blue" BorderBrush ="Red" BorderThickness="2" FontSize = "14" FontFamily =
"Trebuchet MS" Padding="5,3,3,5"/>
```

Замечание. В этом случае ширина элемента будет рассчитываться по содержимому – тексту на кнопке, а расстояние между областью текста и границ элемента определяется свойством Padding.

3. Постройте и запустите приложение. Обратите внимание на новый вид кнопки при ее активации (вводе текста в текстовое поле).

Замечание. Для того, чтобы придать новый вид и другим кнопкам можно просто скопировать новый свойства для каждой кнопке. Но для автоматизации форматирования особенно при добавлении новых кнопок рекомендуется использовать стили, которые, как правило, задаются в словаре ресурсов. После определения ресурса можно ссылаться на ресурс, который должен быть использован для значения свойства с помощью синтаксиса расширения разметки ресурсов, задающего имя ключа.

4. В представлении XAML над объявлением `<DockPanel>` создайте словарь ресурсов как секцию окна (*Windows.Resources)*:

```
<Window.Resources>
</Window.Resources>
```

5. В секции `<Window.Resources>`:
   a. Укажите атрибут **SolidColorBrush** в качестве ресурса (не забудьте, что каждый ресурс в словаре ресурсов должен иметь уникальный ключ) нужного цвета (**Gold**) для элементов управления, которые могут использовать цвет для своих свойств:

```
<SolidColorBrush x:Key="MyBrush" Color="Gold"/>
```

   b. Далее укажите элемент **<Style>**, в котором установите свойству **TargetType** тип элемента, для которого создается стиль, а затем в дочерних элементах **Setter** с помощью атрибута **Property** установите значения свойств форматирования (обратите внимание как задается свойство **Foreground** – с помощью ссылки на ранее установленный ресурс `x:Key="MyBrush"`):

```
<Style TargetType="Button">
    <Setter Property="Background" Value="Blue"/>
    <Setter Property="FontSize" Value="14"/>
    <Setter Property="Foreground" Value="{StaticResource MyBrush}"/>
    <Setter Property="FontFamily" Value="Trebuchet MS"/>
    <Setter Property="Margin" Value="10,15"/>
    <Setter Property ="BorderBrush" Value="Red"/>
    <Setter Property="BorderThickness" Value="2"/>
    <Setter Property="Padding" Value="5,3,3,5"/>
</Style>
```

6. Указанные в стиле свойства удалите из кода XAML кнопки **setBut**.
7. Добавьте в описание текстового поля и текстового блока указание свойства цвета фона со ссылкой на ресурс:

```
Background="{StaticResource MyBrush}"
```

8. Постройте и запустите приложение. Проверьте, что все кнопки главного окна стали согласно указанного стиля. Обратите внимание, что кнопка второго окна осталось прежней.

Замечание. Стили можно установить для отдельного элемента, для всего окна (как сделано выше), а также для всего приложения (в файле App.xaml в теге `<Application.Resources>`).

9. Для придания общего стилизованного вида всех окон приложения создайте общий ресурс, действующий во всем приложении, для этого:
   a. В файл App.xaml в теге `<Application.Resources>` добавьте код, создающий ресурс – кисть линейной градиентной заливки (цвета можете указать на ваше усмотрение):

```
<Application.Resources>
    <LinearGradientBrush x:Key="gradientStyle" StartPoint="0,0" EndPoint="0,1">
        <GradientStop Color="Gray" Offset="0" />
        <GradientStop Color="Green" Offset="0.5" />
        <GradientStop Color="LawnGreen" Offset="1" />
    </LinearGradientBrush>
</Application.Resources>
```

   b. В файлах XAML главного окна и второго окна добавьте после установки размеров окон свойство цвета фона и присвойте ему ресурс градиентной заливки:

```
Background="{StaticResource gradientStyle}"
```

Замечание. Кисть LinearGradientBrush закрашивает область линейным градиентом. Линейный градиент смешивает два или более цвета в линии, оси градиента. Для указания цветов в градиенте и их позиций используются объекты GradientStop. **LinearGradientBrush** позволяет создавать смешанное заполнение, которое представляет собой переход от одного цвета к другому. Чтобы создать горизонтальное заполнение сверху вниз, можно указать начальную точку (0,0) левого верхнего угла, а конечную точку — (0,1), представляющую левый нижний угол.

Замечание. Если два элемента, например, TextBlock должны совместно использовать некоторые свойства, но один из них должен иметь и дополнительные свойства, то реализовать это можно путем создания нового стиля на основе первого стиля.

10. Добавьте в раздел ресурса главного окна новый стиль для элемента статусной панели, задающий начальные свойства:

```
<Style TargetType="StatusBarItem">
    <Setter Property="FontSize" Value="11" />
    <Setter Property="FontFamily" Value="Trebuchet MS"/>
    <Setter Property="Background" Value="Gainsboro"/>
</Style>
```

11. Далее добавьте еще стиль, расширяющий предыдущий, используя атрибут **BasedOn**:

```
<Style BasedOn="{StaticResource {x:Type StatusBarItem}}"
      TargetType="StatusBarItem"
       x:Key="headerText">
    <Setter Property="FontSize" Value="15" />
    <Setter Property="FontWeight" Value="Bold" />
</Style>
```

12. Постройте и запустите приложение. Проверьте внесенные изменения во внешний вид приложения.

Упражнение 2. Использование шаблонов для создания элементов

В данном упражнении будет переработан пользовательский интерфейс второго окна с целью придания элементам управления привлекательного вида с использованием шаблонов.

Замечание. Шаблон позволяет полностью подменить визуальное дерево элемента, сохранив при этом всю функциональность. Структуру и внешний вид элемента управления можно изменить, определив новый шаблон ControlTemplate для элемента управления.

1. Откройте второе окно MyWindow в режиме разметки XAML.
2. Добавьте в окно раздел ресурсы: `<Window.Resources></Window.Resources>`
3. В ресурсе объявите стиль для кнопок со следующими свойствами (если OverridesDefaultStyle – true, то стиль по умолчанию не будет использоваться):

```
<Window.Resources>
    <Style TargetType="Button">
        <Setter Property="OverridesDefaultStyle" Value="True"/>
        <Setter Property="MinHeight" Value="50"/>
        <Setter Property="MinWidth" Value="50"/>
        <Setter Property="Template">
```

4. Далее укажите в теге `<Setter.Value> </Setter.Value>` шаблон элемента управления:

```
            <Setter.Value>
                <ControlTemplate TargetType="Button">
                    <Grid>
                        <Ellipse Fill="Blue"/>
                        <ContentPresenter HorizontalAlignment="Center"
                                          VerticalAlignment="Center"/>
                    </Grid>
                </ControlTemplate>
            </Setter.Value>
        </Setter>
    </Style>
</Window.Resources>
```

5. Удалите установку размеров кнопки и визуально проверьте, что размеры установились как минимально-допустимые.
6. Добавьте снова свойства высоты и ширины кнопки, например `Height="70" Width="70"` и проверьте, что кнопка увеличилась.
7. Постройте и запустите приложение. Проверьте изменений вид кнопки во втором окне.
8. Добавьте новую кнопку в сетку Grid текущего окна – добавленная кнопка должна с принять внешний вид согласно шаблону. Удалите новую кнопку.

Замечание. В данном случае кнопка представляет собой заполненный голубой эллипс. Можно задавать любой вид элемента, например, следующий код разметки создаст ромб с радиальной заливкой (можете изменить цвета или градус наклона на ваше усмотрение):

```
<Rectangle>
   <Rectangle.Fill>
       <RadialGradientBrush>
           <GradientStop Color="Blue" Offset="0" />
           <GradientStop Color="BlueViolet" Offset="1" />
       </RadialGradientBrush>
   </Rectangle.Fill>
   <Rectangle.LayoutTransform>
       <RotateTransform Angle="45" />
   </Rectangle.LayoutTransform>
```

```
</Rectangle>
```

9. Вставьте указанный код вместо `<Ellipse Fill="Blue"/>`, постройте и проверьте изменение внешнего вида.
10. Оставьте внешний вид кнопки на ваше усмотрение.

Практическое занятие 6. Использование триггеров и анимации для создания интерактивного интерфейса

Упражнение 1. Использование триггеров для улучшения внешнего вида интерфейса

Замечание. Триггеры позволяют декларативно реализовывать изменения свойств в соответствии с изменениями других свойств в коде обработки событий. Самым распространенным типом триггера является триггер свойств. Он отслеживает значение свойства элемента управления (таких как **IsFocused**, **IsMouseOver** и **IsPressed** класса **Control**), определенного в свойстве **Property**. Когда значение указанного свойства соответствует значению **Value**, объект **Trigger** активизируется и в результате можно создать эффекты наведения курсора мыши и получения фокуса. Триггеры связываются со стилями через коллекцию **Style.Triggers**.

В этой части упражнении вы примените триггер-свойства для реализации интерактивного интерфейса пользователя – при активации (свойство **IsFocused)** текстового поля появляются визуальные эффекты: увеличивается размер шрифта текста, появляется рамка синего цвета, а сам текст становится красным.

1. Добавьте в представлении XAML главного окна стиль, устанавливающий тип и размер шрифта, а также ориентация текста по вертикали (он будет применен далее для элемента **TextBox)**:

```
    <Style x:Key="FontTextBox">
        <Style.Setters>
            <Setter Property="Control.FontFamily" Value="Calibri"></Setter>
            <Setter Property="Control.FontSize" Value="14"></Setter>
            <Setter Property="TextBox.VerticalContentAlignment"
Value="Center"></Setter>
        </Style.Setters>
```

2. Далее добавьте секцию **Style.Triggers** для настройки триггера свойств (свойство **IsFocused)**:

```
        <Style.Triggers>
            <Trigger Property="Control.IsFocused" Value="True">
                <Setter Property="Control.Foreground" Value="Red"></Setter>
                <Setter Property="TextBox.BorderBrush" Value="Blue"></Setter>
                <Setter Property="Control.FontSize" Value="18"></Setter>
                <Setter Property="TextBox.BorderThickness" Value="3"></Setter>
            </Trigger>
        </Style.Triggers>
    </Style>
```

3. В разметке текстового поля (`x:Name="setText"`) удалите свойство высоты и ширины (если они есть) элемента, но добавьте атрибут установки созданного стиля:

```
Style="{StaticResource ResourceKey=FontTextBox}"
```

4. Постройте и проверьте, что при получении текстового поля фокуса визуально меняется текст, а при потере фокуса – форматирование приходит в исходное состояние.

Замечание. Допускается создавать множество триггеров, которые могут применяться к одному и тому же элементу одновременно.

5. Добавьте секцию **Style.Triggers** еще один триггер, реагирующий на наведение мыши на текстовое поле (**IsMouseOver**):

```
<Trigger Property="Control.IsMouseOver" Value="True">
    <Setter Property="Control.Foreground" Value="Red"></Setter>
    <Setter Property="TextBox.BorderBrush" Value="Blue"></Setter>
    <Setter Property="Control.FontSize" Value="18"></Setter>
    <Setter Property="TextBox.BorderThickness" Value="3"></Setter>
</Trigger>
```

6. Постройте приложение и проверьте, что ожидаемый эффект проявляется в обоих случаях: или при получении фокуса или при наведении мыши.

Замечание. Чтобы создать триггер, срабатывающий только при соблюдении сразу нескольких условий, можно воспользоваться мультитриггером – классом **MutliTrigger**. Этот класс имеет коллекцию **Conditions**, которая позволяет определять цепочки комбинаций свойств и значений.

7. Удалите (или закомментируйте) оба созданных ранее триггера и добавьте следующий мультитриггер:

```
<MultiTrigger>
    <MultiTrigger.Conditions>
        <Condition Property="Control.IsMouseOver" Value="True" />
        <Condition Property="Control.IsFocused" Value="True" />
    </MultiTrigger.Conditions>
    <MultiTrigger.Setters>
        <Setter Property="Control.Foreground" Value="Red"></Setter>
        <Setter Property="TextBox.BorderBrush" Value="Blue"></Setter>
        <Setter Property="Control.FontSize" Value="18"></Setter>
        <Setter Property="TextBox.BorderThickness" Value="3"></Setter>
    </MultiTrigger.Setters>
</MultiTrigger>
```

8. Постройте приложение и проверьте, что ожидаемый эффект проявляется только в одновременном наступлении обоих событий: при получении фокуса и при наведении мыши.

9. Добавьте в стиль кнопки элемент **Trigger**, который изменяет цвет шрифта (свойство Foreground) кнопки при наведении на нее мыши. Результата может быть следующим:

```
<Style.Triggers>
    <Trigger Property="IsMouseOver" Value="true">
        <Setter Property = "Foreground" Value="OrangeRed"/>
    </Trigger>
</Style.Triggers>
```

10. Постройте приложение и проверьте, что ожидаемый эффект.

Упражнение 2. Применение аанимационных эффектов в интерфейсе приложения

В этом упражнении вы добавите анимационный эффект – при наведении мыши на кнопку она будет постепенно увеличиваться в размерах.

Замечание. Поскольку анимация изменяет значения свойств, для различных типов свойств существуют различные типы анимации (они представлены в пространстве имен System.Windows.Media.Animation).

В данном упражнении рассматривается анимация свойства, которое принимает значение Double, например свойства Width элемента.

Замечание. Анимация запускается как реакция на определенное событие или при достижении свойством зависимости определенного значения, что очень удобно в ситуации, когда необходимо запустить анимацию, а подходящего события нет.

Использование триггера свойства

В первой части упражнения рассматривается запуск анимации на изменение свойства – для применения такого подхода необходим триггер свойств, для которого устанавливаются свойства **Trigger.EnterActions** (получает коллекцию объектов TriggerAction, применяемую, когда срабатывает триггер**)** и **Trigger.ExitActions** (получает коллекцию объектов TriggerAction, применяемую, когда объект триггера становится неактивным**)**.

1. Добавьте в разметку стиля кнопки второго окна (MyWindow) секцию **Style.Triggers** для настройки триггера свойств (свойство **IsMouseOver**):

```
<Style.Triggers>
    <Trigger Property="IsMouseOver" Value="True">
```

2. Далее добавьте атрибут **Trigger.EnterActions** с указанием объекта **BeginStoryboard** (действие, запускающее анимацию) в котором объекту Storyboard (служит средством применения анимации к свойствам) укажите `Duration="0:0:1"` и настройте объект анимации DoubleAnimation, изменяющий размер шрифта (до 20), ширину и высоту элемента (оба до 100):

```
<Style.Triggers>
    <Trigger Property="IsMouseOver" Value="True">
        <Trigger.EnterActions>
            <BeginStoryboard Name="Storyboard">
               <Storyboard Duration="0:0:1">
                  <DoubleAnimation Storyboard.TargetProperty="FontSize" To="20" />
                  <DoubleAnimation Storyboard.TargetProperty="Width" To="100"/>
                  <DoubleAnimation Storyboard.TargetProperty="Height" To="100" />
              </Storyboard>
            </BeginStoryboard>
        </Trigger.EnterActions>
```

3. Для выполнения действий, когда объект триггера (кнопка) станет неактивным добавьте объект **Trigger.ExitActions** с указанием объекта `StopStoryboard`:

```
            <Trigger.ExitActions>
                <StopStoryboard BeginStoryboardName="Storyboard" />
            </Trigger.ExitActions>
        </Trigger>
    </Style.Triggers>
```

4. Постройте и протестируйте приложение. Проверьте, что во втором окне при наведении на кнопку указателя мыши увеличивается ее размер и размер текста, а при убирании указателя мыши кнопка приходит в исходное состояние.

5. Добавьте новую кнопку в сетку Grid текущего окна. Запустите приложение – добавленная кнопка должна не только принять внешний вид согласно шаблону, но и анимироваться также. Закройте приложение и удалите новую кнопку.

Использование триггера события

Замечание. Если обычный триггер ожидает изменения свойства, то триггер события (EventTrigger) ожидает возникновения конкретного события. Триггер событий требует предоставления последовательности действий, модифицирующих элемент управления, эти действия и используются для применения анимации.

В этой части упражнения рассматривается применение той же анимации, но запущенной с помощью триггера событий.

1. Удалите (или закомментируйте реализованный триггер свойств).
2. Добавьте триггер событий, который ожидает события **MouseEnter** и затем выполняет анимацию размеров кнопки:

```
<EventTrigger RoutedEvent="Mouse.MouseEnter">
    <EventTrigger.Actions>
        <BeginStoryboard>
            <Storyboard Duration="0:0:1">
                <DoubleAnimation Storyboard.TargetProperty="FontSize" To="20" />
                <DoubleAnimation Storyboard.TargetProperty="Width" To="100"/>
                <DoubleAnimation Storyboard.TargetProperty="Height" To="100" />
            </Storyboard>
        </BeginStoryboard>
    </EventTrigger.Actions>
</EventTrigger>
```

3. Постройте и протестируйте приложение. Проверьте, что во втором окне при наведении на кнопку указателя мыши увеличивается ее размер и размер текста, но при убирании указателя мыши кнопка не приходит в исходное состояние.

Замечание. В отличие от триггеров свойств, действие триггеров событий необходимо обращать, чтобы элемент возвращался в свое исходное состояние (после завершения анимация остается активной и тем самым сохраняет за свойством последнее значение).

4. Чтобы вернуть размеры кнопки в исходное состояние добавьте после первого событийного триггера новый триггер событий, который будет реагировать на событие **MouseLeave** и уменьшать размеры кнопки до исходного состояния:

```
<EventTrigger RoutedEvent="Mouse.MouseLeave">
    <EventTrigger.Actions>
        <BeginStoryboard>
            <Storyboard Duration="0:0:1">
                <DoubleAnimation Storyboard.TargetProperty="FontSize />
                <DoubleAnimation Storyboard.TargetProperty="Width"/>
                <DoubleAnimation Storyboard.TargetProperty="Height"/>
            </Storyboard>
        </BeginStoryboard>
    </EventTrigger.Actions>
</EventTrigger>
```

Замечание. Обратите внимание, что указывать целевые размеры в данном случае не требуется: если они не указаны, то предполагается, что кнопке нужно вернуть первоначальный размер, который был перед выполнением анимации:

5. Постройте и протестируйте приложение. Проверьте, что во втором окне при наведении на кнопку указателя мыши увеличивается ее размер и размер текста, а при убирании указателя мыши кнопка плавно приходит в исходное состояние.

Использование триггера в составе шаблона

Замечание. Триггеры можно указывать в шаблоне, разрабатываемом для элемента управления.

1. После настройки сетки в объекте `ControlTemplate` добавьте секцию `<ControlTemplate.Triggers> </ControlTemplate.Triggers>` в которую перенесите ранее созданный триггер, например, триггер свойств:

```
<ControlTemplate TargetType="Button">
  <Grid>
        ...
  </Grid>
  <ControlTemplate.Triggers>
     <Trigger Property="IsMouseOver" Value="True">
        <Trigger.EnterActions>
           <BeginStoryboard Name="Storyboard">
              <Storyboard Duration="0:0:1">
                <DoubleAnimation Storyboard.TargetProperty="FontSize" To="20" />
                <DoubleAnimation Storyboard.TargetProperty="Width" To="100"/>
                <DoubleAnimation Storyboard.TargetProperty="Height" To="100" />
              </Storyboard>
           </BeginStoryboard>
        </Trigger.EnterActions>
        <Trigger.ExitActions>
            <StopStoryboard BeginStoryboardName="Storyboard" />
        </Trigger.ExitActions>
     </Trigger>
  </ControlTemplate.Triggers>
</ControlTemplate>
```

2. Постройте и протестируйте приложение. Проверьте, что функциональность его не изменилась.

Практическое занятие 7. Использование словарей ресурсов в приложении

Замечание. Приложение WPF может иметь несколько ресурсов пользовательского интерфейса), которые применяются во всем приложении. В совокупности этот набор ресурсов может рассматриваться в качестве темы для приложения. WPF предоставляет поддержку для упаковки ресурсов пользовательского интерфейса в качестве темы с помощью словаря ресурсов, который инкапсулируется в качестве класса ResourceDictionary.

Упражнение 1. Добавление готовых тем в приложение

В этом упражнении вы добавите в приложение тему, согласно которой изменится интерфейс приложения.

Замечание. Темы WPF определяются с помощью механизмов стилизации и шаблонов, которые WPF предоставляет для настройки отображения любого элемента. Ресурсы темы

хранятся в словарях внедренных ресурсов. Готовые темы для WPF приложений можно скачать, например, по адресу:

http://wpf.codeplex.com/wikipage?title=WPF%20Themes&referringTitle=Home

1. Добавьте XAML-файл требуемой темы в проект: в обозревателе решений щелкните правой кнопкой мыши на проекте, в контекстном меню выберите **Добавить -> существующий элемент** и выберите соответствующий файл XAML (в папке WPF Themes), например, ShinyBlue.xaml.
2. В файле App.xaml, добавьте ссылку на желаемую тему в разделе **Application.Resources**:

```
<Application.Resources>
    <ResourceDictionary Source="ShinyBlue.xaml"/>
</Application.Resources>
```

3. Постройте и протестируйте приложение. Добавьте другую тему по своему усмотрению.

Замечание. Тема является последним местом поиска стиля элемента. Обычно поиск начинается с прохода вверх по дереву элементов в поисках соответствующего ресурса, а затем переходит в коллекции ресурсов приложений. Это дает разработчикам приложения возможность переопределить стиль для любого объекта на уровне дерева или приложения до достижения темы.

4. Указанный в стиле темы цвет надписи чекбокса не позволяет прочитать текст. Есть смысл явно указать данный атрибут в настройке элемента, поэтому добавьте в разметку статусной панели в элемент `StatusBarItem x:Name="stb4"` для элемента `<CheckBox x:Name="check"` свойство `Foreground="DarkBlue"`– тем самым вы заместите стиль, указанный в файле темы.
5. Добавьте в XAML-код главного окна тег `<Window.Resources>` и в нем укажите ранее используемый стиль для текстового поля – тем самым вы также заместите стиль, указанный в файле темы:

```
<Window.Resources>
    <Style TargetType="TextBox">
        <Style.Setters>
            <Setter Property="Control.FontFamily" Value="Calibri"></Setter>
            <Setter Property="Control.FontSize" Value="14"></Setter>
            <Setter Property="TextBox.VerticalContentAlignment" Value="Top"></Setter>
        </Style.Setters>
        <Style.Triggers>
            <Trigger Property="Control.IsFocused" Value="True">
                <Setter Property="Control.Foreground" Value="Red"></Setter>
                <Setter Property="TextBox.BorderBrush" Value="Blue"></Setter>
                <Setter Property="Control.FontSize" Value="18"></Setter>
                <Setter Property="TextBox.BorderThickness" Value="3"></Setter>
            </Trigger>
            <Trigger Property="Control.IsMouseOver" Value="True">
                <Setter Property="Control.Foreground" Value="Red"></Setter>
                <Setter Property="TextBox.BorderBrush" Value="Blue"></Setter>
                <Setter Property="Control.FontSize" Value="18"></Setter>
                <Setter Property="TextBox.BorderThickness" Value="3"></Setter>
            </Trigger>
        </Style.Triggers>
    </Style>
```

```
</Window.Resources>
```

6. В разметке текстового поля замените свойство `Height="23"` на минимально допустимое `MinHeight="23"` – так будет возможно увеличение высоты поля при увеличении размера текста.
7. Постройте и протестируйте приложение. Проверьте, что внешний вид соответствует теме, но текстовое поле главного окна ведет себя соответственно указанного вами стиля и надпись чекбокса в статусной панели теперь читаема.

Упражнение 2. Создание и использование словаря ресурсов

Замечание. Словари ресурсов можно определить в виде отдельных файлов, которые позволяют повторно использовать настройки для нескольких приложений.

В этом упражнении вы создадите словарь ресурсов и примените его в своем приложении.

1. Добавьте XAML-файл словаря ресурсов в проект: в обозревателе решений щелкните правой кнопкой мыши на проекте, в контекстном меню выберите **Добавить** -> **Создать элемент** и выберите **Словарь ресурсов (WPF)**, укажите имя файла, например, MyDictionary.xaml.
2. В открывшийся файл MyDictionary.xaml в раздел `<ResourceDictionary` добавьте последовательно следующие элементы, которые были использованы в предыдущих упражнениях курса:

Ресурс градиентной заливки (предполагается, что будет применяться для фона окон):

```
<LinearGradientBrush x:Key="gradientStyle" StartPoint="0,0" EndPoint="0,1">
    <GradientStop Color="Gray" Offset="0" />
    <GradientStop Color="Green" Offset="0.5" />
    <GradientStop Color="LawnGreen" Offset="1" />
 </LinearGradientBrush>
```

Шаблон для кнопок (предполагается, что будет применяться для тех кнопок, которые укажут данный шаблон с помощью ключа):

```
<ControlTemplate x:Key="customButtonTemplate" TargetType="Button">
    <Grid>
        <Rectangle>
            <Rectangle.Fill>
                <RadialGradientBrush >
                    <GradientStop Color="Blue" Offset="0" />
                    <GradientStop Color="BlueViolet" Offset="1" />
                </RadialGradientBrush>
            </Rectangle.Fill>
            <Rectangle.LayoutTransform>
                <RotateTransform x:Name="rotateTrans" Angle="45" />
            </Rectangle.LayoutTransform>
        </Rectangle>
        <ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center"/>
    </Grid>
    <ControlTemplate.Triggers>
        <Trigger Property="IsMouseOver" Value="True">
            <Trigger.EnterActions>
                <BeginStoryboard Name="Storyboard">
                    <Storyboard Duration="0:0:1">
                        <DoubleAnimation Storyboard.TargetProperty="FontSize" To="20"/>
                        <DoubleAnimation Storyboard.TargetProperty="Width" To="100"/>
                        <DoubleAnimation Storyboard.TargetProperty="Height" To="100"/>
                    </Storyboard>
                </BeginStoryboard>
```

```
            </Trigger.EnterActions>
            <Trigger.ExitActions>
                <StopStoryboard BeginStoryboardName="Storyboard" />
            </Trigger.ExitActions>
        </Trigger>
    </ControlTemplate.Triggers>
</ControlTemplate>
```

Стиль для текстовых полей:

```
<Style TargetType="TextBox">
    <Style.Setters>
        <Setter Property="Control.FontFamily" Value="Calibri"></Setter>
        <Setter Property="Control.FontSize" Value="14"></Setter>
        <Setter Property="TextBox.VerticalContentAlignment" Value="Top"></Setter>
    </Style.Setters>
    <Style.Triggers>
        <Trigger Property="Control.IsFocused" Value="True">
                <Setter Property="Control.Foreground" Value="Red"></Setter>
                <Setter Property="TextBox.BorderBrush" Value="Blue"></Setter>
                <Setter Property="Control.FontSize" Value="18"></Setter>
                <Setter Property="TextBox.BorderThickness" Value="3"></Setter>
            </Trigger>
            <Trigger Property="Control.IsMouseOver" Value="True">
                <Setter Property="Control.Foreground" Value="Red"></Setter>
                <Setter Property="TextBox.BorderBrush" Value="Blue"></Setter>
                <Setter Property="Control.FontSize" Value="18"></Setter>
                <Setter Property="TextBox.BorderThickness" Value="3"></Setter>
            </Trigger>
    </Style.Triggers>
</Style>
```

3. Сохраните измененный файл.
4. В файле App.xaml, добавьте объект `<ResourceDictionary.MergedDictionaries>` и в нем укажите ссылки на желаемую тему и ваш словарь ресурсов (именно в такой последовательности, чтобы ваши стили перекрыли стили стандартной темы):

```
<Application.Resources>
    <ResourceDictionary>
        <ResourceDictionary.MergedDictionaries>
            <ResourceDictionary Source="ShinyBlue.xaml"/>
            <ResourceDictionary Source="MyDictionary.xaml"/>
        </ResourceDictionary.MergedDictionaries>
    </ResourceDictionary>
    </Application.Resources>
```

5. Для применения указанных стилей и шаблонов в словаре ресурсов:
   a. В файлах XAML главного окна и второго окна добавьте после установки размеров окон свойство цвета фона и присвойте ему ресурс градиентной заливки:

      `Background="{StaticResource gradientStyle}"`

   b. В файле XAML второго окна в свойствах кнопки укажите атрибут – ссылку на созданный шаблон `Template="{DynamicResource customButtonTemplate}"`.
6. Постройте и протестируйте приложение. Должны работать настройки вашего словаря ресурсов (поведение и вид текстовых полей обеих окон, а также кнопка второго окна) и стандартной темы (остальные кнопки и другие элементы).

Практическое занятие 8. Привязка данных

Замечание. Привязка данных обеспечивает согласованный способ представления данных и взаимодействия с ними в приложениях. Можно связывать элементы с данными из разнообразных источников данных в форме объектов среда CLR и XML. Элементы управления **ContentControl** (например, Button), элементы управления ItemsControl (например, ListBox и ListView) обладают встроенной возможностью включения гибких стилей для отдельных элементов данных и коллекций элементов данных. Представления сортировки, фильтрации и группировки могут быть организованы поверх данных.

Упражнение 1. Создание привязки данных к объекту-коллекции

В этом упражнении будет показано, как использовать код XAML для создания привязок данных, подключающих данные к элементу управления. Вы внесете изменения в ранее созданный проект и добавите источник данных. Этим источником данных является класс **ObservableCollection<T>**, который представляет коллекцию динамических данных, содержащую экземпляры класса **Student**.

Создание источника данных

1. Откройте итоговое приложение практического занятия №6.
2. Добавьте в Grid второго окна элемент **ListBox**, разместите его на ваше усмотрение, например, укажите следующие атрибуты:

```
<ListBox Height="60" HorizontalAlignment="Left"
         Margin="12,12,0,0" Name="listBox1"
         VerticalAlignment="Top" Width="250"/>
```

3. Добавьте в проект новый файл – класс с именем **Student**.
4. В класс **Student** добавьте два автоматических свойства **StudentName** и **IsEnrolled** (булевого типа – информация, например о том зарегистрирован ли студент на мероприятие или нет), конструктор и свойство только для чтения **FullStudentData**, определяющее полную информацию о студенте:

```
public class Student
{
    public string StudentName { get; set; }
    public bool IsEnrolled { get; set; }

    public Student(string name, bool ch)
    {
        StudentName = name;
        IsEnrolled = ch;
    }

    public string FullStudentData
    {
        get { return StudentName + "\t" + IsEnrolled; }
    }
// или тоже самое, но с помощью синтаксиса лямбда выражения:
// public string FullStudentData => StudentName + "\t" + IsEnrolled;
}
```

Замечание. Можно использовать в качестве источника привязки объект, являющийся коллекцией пользовательских объектов. Каждый объект играет роль источника для одного экземпляра повторной привязки. Класс ObservableCollection<T> является встроенной реализацией коллекции данных, предоставляющей интерфейс **INotifyCollectionChanged**. В этом упражнении демонстрируется создание и привязка коллекции, производной от класса **ObservableCollection<T>**, который является классом коллекции, предоставляющим уведомления при получении и удалении элементов или при обновлении всего списка.

5. В этот же файл добавьте новый класс **StudentList** производный от класса **ObservableCollection<Student>**:

```
public class StudentList : ObservableCollection<Student>
{
    public StudentList()
    {
        Add(new Student("Lorin Kanev", true));
        Add(new Student("Ivan Petrov", true));
        Add(new Student("Sergey Masov", false));
        Add(new Student("Tais Frolova", true));
        Add(new Student("Elena Diva", false));
    }
}
```

6. Подключите требуемое пространство имен System.Collections.ObjectModel.
7. Добавьте во второе окно (MyWindow) в перечень пространств имен ссылку на пространство имен CLR (у вас может быть другое имя пространства имен, в котором объявлены новые классы): `xmlns:Local="clr-namespace:WpfApplication4"`.

Указание привязки ListBox к источнику данных

1. Сделайте коллекцию доступной для привязки тем же способом, который используется для привязки других объектов CLR, для чего создайте экземпляр коллекции в XAML и укажите коллекцию в качестве ресурса (раздел ресурса текущего окна, можно перед закрывающимся тегом `</Window.Resources>`):

```
<Local:StudentList x:Key="studentCollection"/>
```

2. В списке `<ListBox x:Name="listBox1"` выполните привязку к коллекции, добавив атрибуты `ItemsSource` (указывает на коллекцию, содержащую все объекты, которые будут показаны в списке) и `DisplayMemberPath` (задает путь к значению в объекте источника), используя синтаксис привязки данных к необходимому ресурсу:

```
<ListBox x:Name="listBox1" ...
         ItemsSource="{Binding Source={StaticResource studentCollection}}"
         DisplayMemberPath="FullStudentData"/>
```

3. Постройте и запустите приложение. Откройте второе окно и проверьте, что в списке отобразились данные согласно свойству FullStudentData: полное имя и через табуляцию информация о регистрации.

Замечание. При реализации привязки списка к источнику атрибут `DisplayMemberPath` не является обязательным, но при его отсутствии для корректного вывода данных о студенте необходимо будет переопределить метод ToString() сущностного класса Student.

Создание привязки к данным на основе шаблона с преобразователем значений

Замечание. Для визуализации результатов регистрации можно применить вместо просто строковых значений true и false нечто более привлекательное, например маленькие квадраты разного цвета. Для того, чтобы цвет квадрата отражал результат (например, красный – false, зеленый – true) необходимо преобразовать значение булевого свойства во что-то, что примет свойство, работающее с цветом. В этом случае для создания привязки в первую очередь необходимо преобразовать значения с помощью пользовательского преобразователя, реализующего интерфейс **IValueConverter**,

1. Добавьте в проект новый класс с именем **BoolToBrushConverter**. Класс **BoolToBrushConverter** будет представлять собой конвертер, который свяжет логическое значение и свойство кисти.
2. Подключите требуемые пространства имен:

```
using System.Windows.Data;
using System.Windows.Media;
using System.Globalization;
```

1. Для класса **BoolToBrushConverter** укажите атрибут **ValueConversion** и реализуйте методы интерфейса **Convert()** и **ConvertBack()**:

```
[ValueConversion(typeof(bool), typeof(Brush))]
public class BoolToBrushConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter,
CultureInfo culture)
    {
        Brush b = null;
        if (value != null &&
            value.GetType() == typeof(bool))
        {
            b = (bool)value ? Brushes.Green : Brushes.Red;
        }
        return b;
    }
    public object ConvertBack(object value, Type targetType, object
parameter, CultureInfo culture)
    {
        return null;
    }
}
```

2. Постройте проект.

Замечание. При отсутствии особых инструкций по способу отображения сбора данных элемент управления ListBox отображает строковое представление каждого объекта в

базовой коллекции, чтобы устранить эту проблему, необходимо задать шаблоны DataTemplate. В WPF для определения визуального представления данных используется шаблон DataTemplate. То, что будет помещено в DataTemplate, определяет внешний вид данных в отображаемом приложении.

1. Откройте файл MyWindow.xaml.
2. Укажите только что созданный преобразователь в качестве ресурса (раздел ресурса текущего окна, можно перед закрывающимся тегом `</Window.Resources>`):

```
<Local:BoolToBrushConverter x:Key="boolToBrushConverter" />
```

3. После этого добавьте XAML-код создающий требуемый шаблон данных – панель `StackPanel`, содержащую прямоугольник с привязкой к логическим данным и преобразователю, а также текстовый блок с именем студента:

```
<DataTemplate x:Key="listBoxTemplate">
    <StackPanel Orientation="Horizontal">
        <Rectangle Fill="{Binding Path=IsEnrolled, Converter={StaticResource
boolToBrushConverter}}"
                   Height="10"
                   Width="10"
                   Margin="0,0,5,0" />
        <TextBlock Text="{Binding Path=StudentName}" />
    </StackPanel>
</DataTemplate>
```

4. В привязке списка `<ListBox x:Name="listBox1"` удалите атрибут `DisplayMemberPath="FullStudentData"` (он становится не нужен ввиду применения созданного шаблона данных) и вместо него добавьте свойство ItemTemplate со значением созданного шаблона (значение DataTemplate):

```
ItemTemplate="{StaticResource listBoxTemplate}
```

5. Постойте и запустите приложение. Проверьте, что во втором окне элемент управления **ListBox** отображает рядом с именем каждого студента красный или зеленый квадратик, указывающий, зарегистрирован студент или нет.
6. Дополните приложение следующим функционалом: по клику кнопки второго окна в текстовый блок первого окна передаются данные о выбранном в списке студенте и комментарий из текстового поля.

Практическое занятие 9. Использование компонента BackgroundWorker

Упражнение. Работа с компонентом BackgroundWorker

В данном упражнении будет создано приложение, использующее компонент **BackgroundWorker** для реализации функционирования долговременного метода, который должен выполняться в отдельном потоке.

1. Откройте Visual Studio и создайте новый проект WPF Application. Проект откроется с формой по умолчанию с именем **MainWindow** в конструкторе.
2. Перетащите из **Toolbox** на плоскость проекта две кнопки и два элемента *label* разместите их так, как вам будет удобно.
3. В окне **Properties** задайте для этих элементов свойство **Content**:

button1	Start
button2	Cancel
labal1	Cycles:
label2	пусто

4. Создайте новый экземпляр компонента BackgroundWorker:

```
System.ComponentModel.BackgroundWorker aWorker = new
  System.ComponentModel.BackgroundWorker();
```

5. В конструкторе класса окна свойству WorkerSupportsCancellation компонента установите значение True:

```
aWorker.WorkerSupportsCancellation = true;
```

6. Создайте методы для обработки события `DoWork` для запуска фонового процесса и события `RunWorkerCompleted`, которое инициируется при завершении или отмены фонового процесса:

```
private void aWorker_DoWork(object sender,
System.ComponentModel.DoWorkEventArgs e)
        {   }
private void aWorker_RunWorkerCompleted(object sender,
  System.ComponentModel.RunWorkerCompletedEventArgs e)
        {    }
```

7. Добавьте в конструктор окна код перехвата событий в обработчик:

```
aWorker.DoWork += aWorker_DoWork;
  aWorker.RunWorkerCompleted += aWorker_RunWorkerCompleted;
```

8. Добавьте в класс окна код для делегата метода UpdateLabel:

```
private delegate void UpdateDelegate(int i);
        private void UpdateLabel(int i)
        {
            label1.Content = "Cycles: " + i.ToString();
        }
```

9. В обработчик события `DoWork` добавьте код реализации:

```
 for (int i = 0; i <= 50; i++)
            {
                for (int j = 1; j <= 10000000; j++)
                {
                }
                if (aWorker.CancellationPending)
                {
                    e.Cancel = true;
                    return;
                }
                UpdateDelegate update = new UpdateDelegate(UpdateLabel);
                label1.Dispatcher.BeginInvoke(
System.Windows.Threading.DispatcherPriority.Normal, update, i);
```

```
            }
```

10. В обработчик события `RunWorkerCompleted` добавьте код реализации:

```
if (!(e.Cancelled))
                label2.Content = "Run Completed";
            else
                label2.Content = "Run Cancelled";
```

11. Дважды кликните кнопки *Start* и *Cancel* и в обработчиках событий добавьте следующие фрагменты кода:

для `button1_Click`:

```
            aWorker.RunWorkerAsync();
```

для `button2_Click`:

```
aWorker.CancelAsync();
```

12. Постройте и протестируйте приложение.

Практическое занятие 10. Настройка интерфейса пользователя

Упражнение 1. Использование элементов Windows Forms

В этом упражнении будет использованы элементы Windows Forms в приложении WPF. Вы создадите приложение, использующее элемент управления *MaskedTextBox* для сбора телефонных номеров и записи их списка в файл, выбранный с помощью элемента *SaveFileDialogBox*.

1. Откройте Visual Studio и создайте новый проект WPF Application, имя укажите, например, UserForms.
2. Перенесите из Toolbox на плоскость проекта элемент *WindowsFormsHost* и задайте для него размер, примерно соответствующий размеру обычного элемента *TextBox*.

В представлении XAML включите пространства имен System.Windows.Forms.Integration:

```
xmlns:my="clr-
  namespace:System.Windows.Forms.Integration;assembly=WindowsFormsIntegration"
```

и System.Windows.Forms:

```
xmlns:wf="clr-namespace:System.Windows.Forms;assembly=System.Windows.Forms"
```

3. Измените элемент *WindowsFormsHost*, включив в него дочерний элемент *MaskedTextBox*:

```
<WindowsFormsHost Height="19" HorizontalAlignment="Left" Margin="30,12,0,0"
  Name="windowsFormsHost1" VerticalAlignment="Top" Width="129" >
    <wf:MaskedTextBox />
            </WindowsFormsHost>
```

4. Задайте элементу *MaskedTextBox* имя *MaskedTextBox1*, а свойству Mask присвойте значение (000)-000-0000:

```
<wf:MaskedTextBox Name="MaskedTextBox1" Mask="(000)-000-0000"/>
```

5. В код XAML добавьте две кнопки в качестве дополнительных дочерних элементов *Grig*:

```
<Button Height="23" Margin="21,76,125,0" Name="Button1" VerticalAlignment="Top"
  Click="Button1_Click">Add to collection</Button>
<Button Margin="21,0,125,118" Name="Button2" Height="22"
  VerticalAlignment="Bottom" Click="Button2_Click">Save collection to
  file</Button>
```

6. В окне кода (файл MainWindow.xaml.cs) в класс *MainWindow* добавьте переменные, представляющие общий элемент *List* строковых объектов и окно *SaveDialogBox* из Windows Forms:

```
List<string> PhoneNumbers = new List<String>();
System.Windows.Forms.SaveFileDialog aDialog;
```

7. В окне дизайнера дважды кликните кнопку *Add To Collection* для добавления обработчика события `Click` и в теле обработчика укажите следующий код:

```
System.Windows.Forms.MaskedTextBox aBox;
  aBox = (System.Windows.Forms.MaskedTextBox)windowsFormsHost1.Child;
  PhoneNumbers.Add(aBox.Text);
  aBox.Clear();
```

8. Добавьте обработчик события `Click` для кнопки *Save Collection To File* и укажите в теле следующий код:

```
aDialog = new System.Windows.Forms.SaveFileDialog();
  aDialog.Filter = "Text Files | *.txt";
  aDialog.ShowDialog();
  System.IO.StreamWriter myWriter = new
           System.IO.StreamWriter(aDialog.FileName, true);
       foreach (string s in PhoneNumbers)
          myWriter.WriteLine(s);
       myWriter.Close();
```

9. Постройте и запустите приложение. Добавьте в коллекцию несколько телефонных номеров и сохраните ее в текстовый файл. Откройте файл из проводника и проверьте запись телефонных номеров.

Упражнение 2. Построение специального свойства зависимости

В этом упражнении будет создано свойство зависимости для специального элемента управления WPF.

1. Создайте новый проект приложения WPF Visual C# с именем WpfUserControl.
2. Добавьте в проект пользовательский элемент: меню **Project → Add New Item** (Проект --> Добавить новый элемент), укажите в качестве шаблона User Control (WPF) (Пользовательский элемент управления (WPF)), и назначьте ему имя ShowNumberControl.xaml.

Как и окно, типы WPF UserControl имеют ассоциированный с ними файл XAML и связанный файл кода.

3. Добавьте в **Grid** элемент управления **Label**:

```
 <Grid>
     <Label Name="numberDisplay" Height="50" Width="200"
  Background="LightBlue"></Label>
```

```
    </Grid>
```

4. В файле кода этого элемента управления создайте свойство .NET, которое упаковывает **int** и устанавливает новое значение для свойства **Content** элемента **Label**:

```
private int currNumber = 0;
public int CurrentNumber
{
      get { return currNumber; }
      set
      {
           currNumber = value;
           // Передаем в метку текущее значение
           numberDisplay.Content = CurrentNumber.ToString();
      }
}
```

5. В коде разметки XAML окна (MainWindow.xaml) объявите экземпляр специального элемента управления внутри диспетчера компоновки и поскольку специальный элемент управления не является частью основного стека сборок WPF, потребуется определить специальное пространство имен XML, которое отобразится на этот элемент:

```
<Window x:Class="WpfUserControl.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:myCtrls="clr-namespace:WpfUserControl"
        Title="MainWindow" Height="350" Width="525">
    <Grid>
        <myCtrls:ShowNumberControl x:Name="mSNC" CurrentNumber="100">

        </myCtrls:ShowNumberControl>
    </Grid>
</Window>
```

6. Добавьте в раздел <myCtrls:ShowNumberControl> код для применения объекта анимации к свойству **CurrentNumber**, чтобы значение изменялось от 100 до 200 за период в 10 секунд:

```
<myCtrls:ShowNumberControl x:Name="mSNC" CurrentNumber="100">
            <myCtrls:ShowNumberControl.Triggers>
                <EventTrigger RoutedEvent="myCtrls:ShowNumberControl.Loaded">
                    <BeginStoryboard>
                        <Storyboard TargetProperty="CurrentNumber">
                            <Int32Animation From="100" To="200"
  Duration="0:0:10"></Int32Animation>
                        </Storyboard>
                    </BeginStoryboard>
                </EventTrigger>
```

```
            </myCtrls:ShowNumberControl.Triggers>
</myCtrls:ShowNumberControl>
```

7. Постройте и запустите приложение. Объект анимации будет проигнорирован.

Причина в том, что свойство **CurrentNumber** не зарегистрировано как свойство зависимости и объект анимации не сможет найти правильную цель.

8. Откройте файл кода специального элемента управления и полностью закомментируйте текущую логику свойства (включая приватное поле).
9. Теперь поместите курсор внутрь области класса и введите фрагмент кода **propdp**. После ввода **propdp** нажмите клавишу <**Tab**>. Фрагмент кода развернется, предоставив базовый скелет свойства зависимости.
10. Замените шаблонное имя свойства на **CurrentNumber** и укажите класс **ShowNumberControl** для оператора **typeof**:

```
public int CurrentNumber
        {
            get { return (int)GetValue(CurrentNumberProperty); }
            set { SetValue(CurrentNumberProperty, value); }
        }

        // Using a DependencyProperty as the backing store for CurrentNumber.
  This enables animation, styling, binding, etc...
        public static readonly DependencyProperty CurrentNumberProperty =
            DependencyProperty.Register("CurrentNumber", typeof(int),
  typeof(ShowNumberControl), new UIPropertyMetadata(0));
```

Данный фрагмент кода регистрирует свойство встроенным способом, а не в статическом конструкторе. Обратите внимание, что объект **UIPropertyMetadata** используется для определения целого значения по умолчанию (0) вместо более сложного объекта **FrameworkPropertyMetadata**.

Добавление процедуры проверки достоверности данных

1. Реализуйте функции для выполнения логики проверки достоверности: требуется обеспечить, чтобы значение свойства **CurrentNumber** находилось в пределах от 0 до 500.
    a. Для этого добавьте финальный аргумент к методу **DependencyProperty.Register()** типа ValidateValueCallback, указывающий на метод по имени **ValidateCurrentNumber()**. ValidateValueCallback — это делегат, который может только указывать на методы, возвращающие bool и принимающие object в качестве единственного аргумента. Этот object представляет новое значение, которое присваивается.

```
// Using a DependencyProperty as the backing store for CurrentNumber.  This
  enables animation, styling, binding, etc...
        public static readonly DependencyProperty CurrentNumberProperty =
            DependencyProperty.Register("CurrentNumber", typeof(int),
            typeof(ShowNumberControl), new UIPropertyMetadata(100),
            new ValidateValueCallback(ValidateCurrentNumber));
```

b. Реализуйте **ValidateCurrentNumber()** для возврата true, если входящее значение находится в заданном диапазоне, и false — в противном случае:

```
public static bool ValidateCurrentNumber(object value)
{
    if (Convert.ToInt32(value) >= 0 && Convert.ToInt32(value) <= 500)
        return true;
    else
        return false;
}
```

Реакция на изменение свойства

1. Для реализации реакции на изменение свойства задайте второй аргумент конструктора **UIPropertyMetadata**, которым является **PropertyChangedCallback**. Этот делегат может указывать на любой метод, принимающий DependencyObject в качестве первого параметра и DependencyPropertyChangeEventArgs — в качестве второго:

```
public static readonly DependencyProperty CurrentNumberProperty =
            DependencyProperty.Register("CurrentNumber", typeof(int),
            typeof(ShowNumberControl),
            new UIPropertyMetadata(100,
                new PropertyChangedCallback(CurrentNumberChanged)),
            new ValidateValueCallback(ValidateCurrentNumber));
```

2. Целью внутри метода **CurrentNumberChanged()** является изменение свойства **Content** объекта **Label** на новое значение, присвоенное свойством **CurrentNumber**. Новое значение можно найти, используя входные аргументы события. Реализуйте метод **CurrentNumberChanged(),** который изменит свойство **Content** объекта **Label**:

```
private static void CurrentNumberChanged(DependencyObject depObj,
    DependencyPropertyChangedEventArgs args)
{
    ShowNumberControl s = (ShowNumberControl)depObj;
    Label theLabel = s.numberDisplay;
    theLabel.Content = args.NewValue.ToString();
}
```

3. Постройте и запустите приложение.

Свойство зависимости **CurrentNumber** теперь может быть целью для стиля WPF, объекта анимации, операций привязки данных и т.д.

Практическое занятие 11. Графика в приложениях WPF

Упражнение 1. Рисование замкнутой фигуры с помощью элемента "Многоугольник"

В этом упражнении выполняется рисование замкнутой фигуры с помощью элемента **Polygon**. Чтобы нарисовать замкнутую фигуру Вы создадите элемент **Polygon** и используете его свойство **Points** для указания вершин фигуры. Автоматически нарисуется линия, соединяющая первую и последнюю точки.

1. Откройте Visual Studio и создайте новый проект WPF Application, имя укажите, например, WpfApplicationPolygon.
2. Откройте файл MainWindow.xaml и замените элемент **Grid** следующим кодом, который нарисует четыре треугольника:

```
<Canvas Height="300" Width="300">
  <!-- Рисует треугольник с голубым наполнением -->
  <Polygon Points="10,110 60,10 110,110"
    Fill="Blue" />

 <!-- Рисует треугольник с голубым наполнением и черным контуром -->
  <Polygon Points="10,110 60,10 110,110"
    Fill="Blue"
    Stroke="Black" StrokeThickness="4"
    Canvas.Top="150" />

  <!-- Рисует еще треугольник с голубым наполнением -->
  <Polygon Points="10,110 110,110 110,10"
    Fill="Blue"
    Canvas.Left="150" />

  <!-- Рисует треугольник с черным контуром -->
  <Polygon Points="10,110 110,110 110,10"
    Stroke="Black" StrokeThickness="4"
    Canvas.Left="150" Canvas.Top="150" />

</Canvas>
```

Обратите внимание, что в XAML допустимым синтаксисом для точек является список, содержащий пары разделенных пробелом координат x и y.

Элементы многоугольника (и все остальные элементы фигур) можно использовать с любым элементом **Panel** или **Control**, поддерживающим нетекстовое содержимое.

3. Добавьте следующий фрагмент кода, который нарисует две звезды. Обратите внимание на свойство **FillRule**:

```
<!-- Polygon A -->
 <Polygon
   Points="10,100 50,0 90,100 0,35 100,35"
   Fill="Gold"
```

```
    Stroke="Black"
    StrokeThickness="3"
    FillRule="Nonzero"
    Canvas.Left="300"
    Canvas.Top="10">
  </Polygon>

  <!-- Polygon B -->
  <Polygon
    Points="10,100 50,0 90,100 0,35 100,35"
    Fill="Gold"
    Stroke="Black"
    StrokeThickness="3"
    FillRule="EvenOdd"
    Canvas.Left="300"
    Canvas.Top="150">
  </Polygon>
```

4. Постройте и запустите приложение.

Упражнение 2. Создание приложения рисования

В этом упражнении реализуется приложение, позволяющее рисовать. Для этой цели применяется элемент **InkCanvas,** который представляет собой полотно, на котором можно рисовать.

1. Откройте Visual Studio и создайте новый проект WPF Application, имя укажите, например, Ink Pad.
2. Измените размеры окна WPF, установив в окне **Свойства** значение 550 для свойства **Height** и значение 370 для свойства **Width**.
3. Измените заголовок окна (свойство **Title**) WPF на **Ink Pad**.
4. Измените свойство **Background** окна WPF на светлосерый цвет, выбрав в списке **LightGray** в раскрывающемся списке и нажав клавишу ВВОД.

Элемент **InkCanvas,** обеспечивающий рукописный ввод отсутствует на панели элементов. Поэтому его требуется добавить.

5. Чтобы открыть панель элементов, в меню **Вид** выберите пункт **Панель элементов**.
6. Щелкните правой кнопкой мыши панель элементов и выберите пункт **Выбрать элементы**. Откроется диалоговое окно **Выбор элементов панели элементов**.
7. На вкладке **Компоненты WPF** диалогового окна В**ыбор элементов панели элементов** прокрутите список вниз до варианта **InkCanvas** и выберите его, установив флажок. Нажмите кнопку ОК для добавления элемента управления **InkCanvas** на панель элементов.
8. Из панели элементов перетащите в окно WPF элемент управления **InkCanvas**.
9. В окне **Свойства** задайте следующие свойства элемента управления **InkCanvas**.

Свойство	**Значение**
Width	Автоматически (сбросить значение)

Height	Автоматически
HorizontalAlignment	Растяжение (Stretch)
VerticalAlignment	Растяжение

10. Измените цвет элемента управления **InkCanvas** на желтый, задав его свойству **Background** значение *LightYellow*. Фон элемента управления "InkCanvas" во время выполнения будет светло-желтым.
11. Перетащите два элемента управления **Button** в окно WPF под элемент управления **InkCanvas**. Расположите **button1** слева, а **button2** — справа.
12. Выберите **button1** и измените разметку XAML в режиме просмотра XAML, эта разметка устанавливает для текста значение *Clear*:

```
<Button Content="Clear"
```

13. Выберите **button2** и измените свойство **Content** на *Close*.
14. Создайте обработчики событий **Click:**
15. Для кнопки **Clear**:

```
this.inkCanvas1.Strokes.Clear();
```

16. Для кнопки **Close**:

```
this.Close();
```

17. Постройте и запустите приложение. В открывшемся приложении нарисуйте рисунок в элементе управления **InkCanvas**. Если допущена ошибка, можно нажать кнопку **Clear**, чтобы начать заново.
18. Нажмите кнопку **Close**, чтобы закрыть приложение.

Все рисование в итоге представляется в виде штрихов – элементов класса System.Windows.Ink.Stroke и хранится в коллекции **Strokes**, определенной в классе **InkCanvas**.

Упражнение 3. Создание диаграмм с использованием WPF Toolkit

В этом упражнении Вы создадите приложение для построения диаграмм, использующее средства библиотеки WPF Toolkit.

19. Установите библиотеку (скопируйте из папки Практика) **WPF Toolkit**.
20. Откройте Visual Studio и создайте новый проект WPF Application, имя укажите, например, WpfApplicationChart.
21. Добавьте ссылку на сборки System.Windows.Controls.DataVisualization.Toolkit.dll и WPFToolkit.dll.
22. Откройте файл WindowChartSample.xaml и импортируйте на страницу имена
    - "System.Windows.Controls.DataVisualization.toolkit"
    - "System.Windows.Controls.DataVisualization.Charting"

При вводе "`xmlns =`", на Вашу страницу, вы увидите эти пространства имен в Intellisense. Выберите и добавьте их обоих на свою страницу:

```
xmlns:chartingToolkit="clr-
namespace:System.Windows.Controls.DataVisualization.Charting;assembly=System.
Windows.Controls.DataVisualization.Toolkit"
```

23. Импортируйте пространство имен `clr`:

```
    xmlns:WpfApplicationChart="clr-namespace:WpfApplicationChart">
```

24. Установите свойство `Title="Chart"`.
25. С помощью следующего кода разметки XAML в элемент **Grid** добавьте диаграмму и настройте ось Y:

```
<Grid>
    <chartingToolkit:Chart >
         <chartingToolkit:Chart.Axes>
              <chartingToolkit:LinearAxis Orientation="Y" Minimum="0"
Maximum="50" Title="Value" ShowGridLines="True" />
          </chartingToolkit:Chart.Axes>
     </chartingToolkit:Chart>
</Grid>
```

26. Добавьте в проект класс **Model**.
27. В файле Model.cs создайте класс **Data** (со свойствами `Name` и `Value`), который представляет точку на плоскости:

```
using System.Collections.Generic;

namespace WpfApplicationChart
{
    public class Data
    {
        public Data(string name, int value)
        {
            Name = name;
            Value = value;
        }

        public string Name { get; set; }
        public int Value { get; set; }
    }
```

28. В классе **Model** реализуйте хранение данных в списке:

```
using System.Collections.Generic;

namespace WpfApplicationChart
{
...

    class Model
    {
        public IList<Data> Data
        {
            get
            {
                IList<Data> list = new List<Data>
```

```
                                        {
                                            new Data("Иванов", 10),
                                            new Data("Петров", 30),
                                            new Data("Сиборов", 10),
                                            new Data("Кузнецов", 20),
                                            new Data("Павлов", 40)
                                        };
                return list;
            }
        }
    }
}
```

29. В файл Window.xaml добавьте секцию **Window.Resources** элементы ресурсов и свяжите их с моделью:

```
<Window.Resources>
    <WpfApplicationChart:Model x:Key="model" />
</Window.Resources>
```

30. В секцию **Grid** добавьте код разметки, связывающий диаграмму с данными модели:

```
 <Grid DataContext="{Binding Source={StaticResource model}}">
    <chartingToolkit:Chart>
...
            <chartingToolkit:ColumnSeries ItemsSource="{Binding Path=Data}"
                 DependentValuePath="Value" IndependentValuePath="Name" />
    </chartingToolkit:Chart>
</Grid>
```

31. Постройте и запустите приложение.

Практическое занятие 12. Использование параметров приложения

Упражнение 1. Работа с параметрами

В данном упражнении будет создано приложение, использующее параметры. Вы определите параметры в процессе построения приложения, прочитаете параметры, примените их в приложении и реализуете возможность изменять параметры.

1. Откройте Visual Studio и создайте новый проект WPF Application, имя укажите, например, Param.
2. В Solution Explorer разверните узел **Properties** и откройте **Settings.settings**.
3. Добавьте два параметра:

Параметр	**Тип**	**Область действия**	**Значение**
ApplicationName	String	Application	Settings App
BackgroundColor	System.Windows.Media.Color	User	#ff0000ff

4. В представлении XAML добавьте следующий код в элемент Grid, чтобы добавить список ListBox с четырьмя элементами и кнопку Button в пользовательский интерфейс:

```
<ListBox Margin="15,15,0,0" Name="listBox1" Height="78"
  HorizontalAlignment="Left" VerticalAlignment="Top" Width="107">
            <ListBoxItem>Red</ListBoxItem>
            <ListBoxItem>Blue</ListBoxItem>
            <ListBoxItem>Green</ListBoxItem>
            <ListBoxItem>Tomato</ListBoxItem>
        </ListBox>
        <Button Margin="15,106,110,130" Name="button1">Change Background
        Color</Button>
```

5. В конструкторе дважды щелкните элемент button1, чтобы открыть представление кода обработчика по умолчанию для события Click и добавьте в обработчик следующий код:

```
if (!(listBox1.SelectedItem == null))
            {
    String astring = ((ListBoxItem)listBox1.SelectedItem).Content.ToString();
                switch (astring)
                {
                    case "Red":
      Properties.Settings.Default.BackgroundColor = Colors.Red;
                        break;
                    case "Blue":
      Properties.Settings.Default.BackgroundColor = Colors.Blue;
                        break;
                    case "Green":
      Properties.Settings.Default.BackgroundColor = Colors.Green;
                        break;
                    case "Tomato":
      Properties.Settings.Default.BackgroundColor = Colors.Tomato;
                        break;
                }
this.Background = new System.Windows.Media.SolidColorBrush(
Properties.Settings.Default.BackgroundColor);
                Properties.Settings.Default.Save();
            }
```

6. В конструкторе класса `MainWindow` укажите следующий код:

```
InitializeComponent();
            this.Title = Properties.Settings.Default.ApplicationName;
            this.Background = new System.Windows.Media.SolidColorBrush(
Properties.Settings.Default.BackgroundColor);
```

7. Постройте и протестируйте приложение. Проверьте, что в заголовке окна отображается значение параметра ApplicationName - Settings App, а фоновый цвет приложения характеризуется значением параметра BackgroundColor. Измените фоновый цвет, выбрав нужный элемент в ListBox и щелкнув кнопку. Закройте приложение.
8. Запустите приложение еще раз и проверьте, что фоновый цвет такой же, как и при завершении предыдущего сеанса приложения.

Лабораторная работа 1. Разработка приложения WPF

В этой лабораторной работе описывается реализация основных элементов, которые являются общими для большинства приложений WPF:

- определение XAML-кода для разработки пользовательского интерфейса приложения;
- написание кода для построения модели поведения приложения;
- создание элементов управления приложением;
- добавление элементов управления и создание макета, входящих в состав пользовательского интерфейса приложения;
- создание стилей, обеспечивающих единый внешний вид компонентов приложения пользовательского интерфейса;
- связывание пользовательского интерфейса с данными для заполнения его данными и их синхронизации с пользовательским интерфейсом.

В результате данной работы будет создано автономное приложение Windows, которое позволит пользователям просматривать отчеты о расходах выбранных лиц. Приложение будет состоять из нескольких страниц WPF, размещенных в окне, напоминающем браузер.

Упражнение 1. Создание инфраструктуры приложения

В этом упражнении создается инфраструктура приложения, которая включает определение приложения, две страницы приложения и одно изображение.

1. Создайте новый проект приложения WPF Visual C# с именем ExpenseIt.
2. Откройте файл MainWindow.xaml.

Этот XAML-файл представляет главное окно приложения, в котором отображается созданное содержимое страниц. Класс **Window** определяет свойства окна и обрабатывает события, такие как открытие и закрытие окна.

3. Замените элемент **Window** на **NavigationWindow**.

Класс **NavigationWindow** наследует все свойства класса **Window**. Элемент **NavigationWindow** в XAML-файле создает экземпляр класса **NavigationWindow**.

4. Измените следующие свойства элемента **NavigationWindow**: **Title** – "ExpenseIt", **Width** – 525 и **Height –** 350.

5. Удалите элементы **Grid** между тегами **NavigationWindow** – тип "NavigationWindow" не поддерживает непосредственное содержимое.

XAML-код должен выглядеть следующим образом:

```
<NavigationWindow x:Class="ExpenseIt.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="ExpenseIt" Height="350" Width="525">

</NavigationWindow>
```

6. Откройте файл MainWindow.xaml.cs.

Это файл программного кода, в котором содержится код обработки событий, объявленных в файле MainWindow.xaml. В этом файле содержится разделяемый класс окна, определенного в XAML-коде.

7. Измените класс **MainWindow** так, чтобы он наследовался от класса **NavigationWindow**.

Код должен выглядеть следующим образом.

```
using System;
using System.Collections.Generic;
...
using System.Windows.Navigation;

namespace ExpenseIt
{
    public partial class MainWindow : NavigationWindow
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

Добавление файлов в приложение

В этом разделе упражнения в приложение добавляются две страницы и изображение.

1. Добавьте в проект новую страницу WPF (в контекстном меню узла проекта выберите **Добавить** > **Страница**…) с именем ExpenseItHome.xaml.

Эта страница будет отображаться первой при запуске приложения. На ней будет выводиться список, в котором можно будет выбрать человека и просмотреть отчет о его затратах.

2. Откройте файл ExpenseItHome.xaml.
3. Задайте свойству **Title** значение "ExpenseIt – Home".

XAML-код должен выглядеть следующим образом:

```
<Page x:Class="ExpenseIt.ExpenseItHome"
      xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
```

```
      xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
      xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
      mc:Ignorable="d"
      d:DesignHeight="300" d:DesignWidth="300"
  Title="ExpenseIt - Home">

    <Grid>

    </Grid>
</Page>
```

4. Откройте файл MainWindow.xaml.
5. Для объекта **NavigationWindow** задайте свойству **Source** значение "ExpenseItHome.xaml".

В результате ExpenseItHome.xaml будет первой страницей, открываемой при запуске приложения. XAML-код должен выглядеть следующим образом:

```
<NavigationWindow x:Class="ExpenseIt.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="ExpenseIt" Height="350" Width="500" Source="ExpenseItHome.xaml">

</NavigationWindow>
```

6. Добавьте в проект новую страницу WPF с именем ExpenseReportPage.xaml.

На этой странице будет представлен отчет о расходах для человека, выбранного на странице ExpenseItHome.xaml.

7. Откройте файл ExpenseReportPage.xaml.
8. Задайте свойству **Title** значение "ExpenseIt - View Expense".

XAML-код должен выглядеть следующим образом:

```
<Page x:Class="ExpenseIt.ExpenseReportPage"
      xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
      xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
      xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
      mc:Ignorable="d"
      d:DesignHeight="300" d:DesignWidth="300"
  Title="ExpenseIt - View Expense">

    <Grid>

    </Grid>
</Page>
```

9. Откройте и изучите файлы ExpenseItHome.xaml.cs и ExpenseReportPage.xaml.cs.

Эти файлы программного кода обрабатывают логику, реагирующую на действия пользователя.

10. Добавьте в проект файл изображения с именем watermark.png из папки Lab 01m. Можно создать собственное изображение.
11. Постройте и запустите приложение. Пока это пустое приложение, отображается только главная страница.

Упражнение. 2 Реализация пользовательского интерфейса приложения

Создание макета приложения

Макет обеспечивает упорядоченное расположение элементов пользовательского интерфейса, а также управление их размером и положением при изменении размера интерфейса. Обычно макет создается с одним из следующих элементов управления макетом: Canvas, DockPanel, Grid, StackPanel, VirtualizingStackPanel, WrapPanel. Для каждого из этих элементов управления макетом поддерживается специальный тип макета дочерних элементов.

Размер страниц приложения ExpenseIt может быть изменен. На каждой странице будут представлены элементы, которые упорядочены по горизонтали и вертикали рядом с другими элементами. Следовательно, макет **Grid** является подходящим элементом макета для приложения.

В этом упражнении будет создана таблица с одним столбцом, тремя строками и полями шириной 10 пикселей путем добавления определений столбцов и строк в класс **Grid** в файле ExpenseItHome.xaml.

1. Откройте файл ExpenseItHome.xaml.
2. Задайте свойству **Margin** элемента **Grid** значение "10,0,10,10", указывающее ширину полей слева, сверху, справа и снизу соответственно.
3. Добавьте следующий XAML-код между тегами **Grid**, чтобы создать определения строк и столбцов (свойству двух строк **Height** задано значение **Auto**, что означает, что размер строк будет зависеть от их содержимого):

```
<Grid Margin="10,0,10,10">
    <Grid.ColumnDefinitions>
        <ColumnDefinition />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition />
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>
</Grid>
```

Добавление и настройка элементов управления

В этом разделе будет обновлен UI стартовой страницы для отображения списка лиц, для которых можно вывести отчеты о расходах. Чтобы создать этот UI, в файл ExpenseItHome.xaml необходимо будет добавить следующие элементы:

ListBox — отображение списка людей,

Label — заголовок списка,

Button — кнопка для просмотра отчета о расходах для выбранного в списке человека.

Каждый элемент управления необходимо поместить в строку таблицы **Grid**, указав присоединенное свойство **Grid.Row**.

1. Откройте файл ExpenseItHome.xaml.
2. Добавьте следующий XAML-код между тегами **Grid:**

```
<!-- People list -->
<Border Grid.Column="0" Grid.Row="0" Height="35" Padding="5"
Background="#4E87D4">
    <Label VerticalAlignment="Center" Foreground="White">Names</Label>
</Border>
<ListBox Name="peopleListBox" Grid.Column="0" Grid.Row="1">
    <ListBoxItem>Mike</ListBoxItem>
    <ListBoxItem>Lisa</ListBoxItem>
    <ListBoxItem>John</ListBoxItem>
    <ListBoxItem>Mary</ListBoxItem>
</ListBox>

<!-- View report button -->
<Button Grid.Column="0" Grid.Row="2" Margin="0,10,0,0" Width="125"
Height="25" HorizontalAlignment="Right">View</Button>
```

Добавление изображения и заголовка

1. В файл ExpenseItHome.xaml добавьте новый столбец в свойство **ColumnDefinitions** с фиксированной длиной **Width** 230 пикселей:

```
<Grid.ColumnDefinitions>
    <ColumnDefinition Width="230" />
    <ColumnDefinition />
</Grid.ColumnDefinitions>
```

2. Добавьте новую строку в свойство **RowDefinitions**.

```
<Grid.RowDefinitions>
    <RowDefinition/>
    <RowDefinition Height="Auto"/>
    <RowDefinition />
    <RowDefinition Height="Auto"/>
</Grid.RowDefinitions>
```

3. Переместите элементы управления во второй столбец, задав свойству **Grid.Column** значение 1 и переместите каждый элемент управления на одну строку вниз, увеличив значение свойства **Grid.Row** на 1.

```
<Border Grid.Column="1" Grid.Row="1" Height="35" Padding="5"
Background="#4E87D4">
    <Label VerticalAlignment="Center" Foreground="White">Names</Label>
</Border>
<ListBox Name="peopleListBox" Grid.Column="1" Grid.Row="2">
    <ListBoxItem>Mike</ListBoxItem>
```

```
        <ListBoxItem>Lisa</ListBoxItem>
        <ListBoxItem>John</ListBoxItem>
        <ListBoxItem>Mary</ListBoxItem>
    </ListBox>

    <!-- View report button -->
    <Button Grid.Column="1" Grid.Row="3" Margin="0,10,0,0" Width="125"
Height="25" HorizontalAlignment="Right">View</Button>
```

4. В качестве значения свойства **Background** класса **Grid** задайте файл изображения watermark.png.

```
<Grid.Background>
    <ImageBrush ImageSource="watermark.png"/>
</Grid.Background>
```

5. Укажите в свойствах файла изображения (watermark.png) "Действие при сборке": Ресурс (Resource).
6. В заголовке страницы перед элементом **Border** добавьте элемент **Label** с содержимым "View Expense Report":

```
<Label Grid.Column="1" VerticalAlignment="Center" FontFamily="Trebuchet MS"
        FontWeight="Bold" FontSize="18" Foreground="#0066cc">
    View Expense Report
</Label>
```

7. Постройте и запустите приложение. Внешний вид приложения должен быть как на следующем рисунке:

Добавление кода для обработки событий

1. Добавьте обработчик событий **Click** в элемент **Button**.

```
    <!-- View report button -->
    <Button Grid.Column="1" Grid.Row="3" Margin="0,10,0,0" Width="125"
Height="25" HorizontalAlignment="Right" Click="Button_Click">View</Button>
```

2. Откройте файл ExpenseItHome.xaml.cs.
3. Добавьте в обработчик события **Click** код, обеспечивающий переход стартового окна к странице ExpenseReportPage.xaml:

```
private void Button_Click(object sender, RoutedEventArgs e)
{
    // View Expense Report
    ExpenseReportPage expenseReportPage = new ExpenseReportPage();
    this.NavigationService.Navigate(expenseReportPage);
}
```

4. Постройте и запустите приложение. Нажмите кнопку **View**. Отобразится пока пустое окно отчета о затратах. Обратите внимание, что доступна кнопка "назад".

Упражнение 3. Создание пользовательского интерфейса отдельной страницы

На странице ExpenseReportPage.xaml должен отображается отчет о расходах для человека, выбранного на странице ExpenseItHome.xaml. В этом упражнении на страницу ExpenseReportPage.xaml добавляются элементы управления, и для нее создается UI. На нее также добавляется фон и цвета заливки для различных элементов.

1. Откройте файл ExpenseReportPage.xaml.
2. Добавьте следующий XAML-код между тегами **Grid**.

Этот пользовательский интерфейс аналогичен пользовательскому интерфейсу, созданному в файле ExpenseItHome.xaml, за исключением того, что данные отчета отображаются в элементе управления **DataGrid**.

```
<Grid.Background>
    <ImageBrush ImageSource="watermark.png" />
</Grid.Background>
<Grid.ColumnDefinitions>
    <ColumnDefinition Width="230" />
    <ColumnDefinition />
</Grid.ColumnDefinitions>
<Grid.RowDefinitions>
    <RowDefinition Height="Auto" />
    <RowDefinition />
</Grid.RowDefinitions>

<Label Grid.Column="1" VerticalAlignment="Center" FontFamily="Trebuchet MS"
FontWeight="Bold" FontSize="18" Foreground="#0066cc">
    Expense Report For:
</Label>
<Grid Margin="10" Grid.Column="1" Grid.Row="1">

    <Grid.ColumnDefinitions>
        <ColumnDefinition />
        <ColumnDefinition />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
```

```
        <RowDefinition />
    </Grid.RowDefinitions>

    <!-- Name -->
    <StackPanel Grid.Column="0" Grid.ColumnSpan="2" Grid.Row="0"
  Orientation="Horizontal">
        <Label Margin="0,0,0,5" FontWeight="Bold">Name:</Label>
        <Label Margin="0,0,0,5" FontWeight="Bold"></Label>
    </StackPanel>

    <!-- Department -->
    <StackPanel Grid.Column="0" Grid.ColumnSpan="2" Grid.Row="1"
  Orientation="Horizontal">
        <Label Margin="0,0,0,5" FontWeight="Bold">Department:</Label>
        <Label Margin="0,0,0,5" FontWeight="Bold"></Label>
    </StackPanel>

    <Grid Grid.Column="0" Grid.ColumnSpan="2" Grid.Row="2"
  VerticalAlignment="Top"
          HorizontalAlignment="Left">
        <!-- Expense type and Amount table -->
        <DataGrid  AutoGenerateColumns="False" RowHeaderWidth="0" >
            <DataGrid.ColumnHeaderStyle>
                <Style TargetType="{x:Type DataGridColumnHeader}">
                    <Setter Property="Height" Value="35" />
                    <Setter Property="Padding" Value="5" />
                    <Setter Property="Background" Value="#4E87D4" />
                    <Setter Property="Foreground" Value="White" />
                </Style>
            </DataGrid.ColumnHeaderStyle>
            <DataGrid.Columns>
                <DataGridTextColumn Header="ExpenseType" />
                <DataGridTextColumn Header="Amount"  />
            </DataGrid.Columns>
        </DataGrid>
    </Grid>
</Grid>
```

3. Постройте и запустите приложение. Нажмите кнопку **View**. Отобразится страница отчета о затратах. Обратите внимание, что доступна кнопка "назад".

Упражнение 4. Использование стилей

Правильный интерфейс подразумевает одинаковый внешний вид различных элементов для всех элементов одного типа. UI использует стили, чтобы варианты внешнего вида можно было многократно использовать для нескольких элементов. Повторное использование стилей помогает упростить создание XAML-кода и управление им.

В этом упражнении атрибуты, установленные ранее для каждого элемента, заменяются стилями.

1. Откройте файл App.xaml.
2. Добавьте следующий XAML-код между тегами **Application.Resources**.

```
<!-- Header text style -->
<Style x:Key="headerTextStyle">
    <Setter Property="Label.VerticalAlignment" Value="Center"></Setter>
    <Setter Property="Label.FontFamily" Value="Trebuchet MS"></Setter>
    <Setter Property="Label.FontWeight" Value="Bold"></Setter>
    <Setter Property="Label.FontSize" Value="18"></Setter>
    <Setter Property="Label.Foreground" Value="#0066cc"></Setter>
</Style>

<!-- Label style -->
<Style x:Key="labelStyle" TargetType="{x:Type Label}">
    <Setter Property="VerticalAlignment" Value="Top" />
    <Setter Property="HorizontalAlignment" Value="Left" />
    <Setter Property="FontWeight" Value="Bold" />
    <Setter Property="Margin" Value="0,0,0,5" />
</Style>

<!-- DataGrid header style -->
<Style x:Key="columnHeaderStyle" TargetType="{x:Type DataGridColumnHeader}">
    <Setter Property="Height" Value="35" />
    <Setter Property="Padding" Value="5" />
    <Setter Property="Background" Value="#4E87D4" />
    <Setter Property="Foreground" Value="White" />
</Style>

<!-- List header style -->
<Style x:Key="listHeaderStyle" TargetType="{x:Type Border}">
    <Setter Property="Height" Value="35" />
    <Setter Property="Padding" Value="5" />
    <Setter Property="Background" Value="#4E87D4" />
</Style>

<!-- List header text style -->
<Style x:Key="listHeaderTextStyle" TargetType="{x:Type Label}">
    <Setter Property="Foreground" Value="White" />
    <Setter Property="VerticalAlignment" Value="Center" />
    <Setter Property="HorizontalAlignment" Value="Left" />
</Style>

<!-- Button style -->
```

```
<Style x:Key="buttonStyle" TargetType="{x:Type Button}">
    <Setter Property="Width" Value="125" />
    <Setter Property="Height" Value="25" />
    <Setter Property="Margin" Value="0,10,0,0" />
    <Setter Property="HorizontalAlignment" Value="Right" />
</Style>
```

В этом XAML-коде добавляются следующие стили:

- headerTextStyle — форматирование заголовка страницы **Label**.
- labelStyle — форматирование элементов управления **Label**.
- columnHeaderStyle — форматирование **DataGridColumnHeader**.
- listHeaderStyle — форматирование элементов управления заголовка списка **Border**.
- listHeaderTextStyle — форматирование заголовка списка **Label**.
- buttonStyle — форматирование элемента **Button** в файле ExpenseItHome.xaml.

Обратите внимание, что стили представляют собой ресурсы и являются дочерними элементами свойства **Application.Resources**. Здесь стили применяются ко всем элементам в приложении.

3. Откройте файл ExpenseItHome.xaml.
4. Замените все содержимое между элементами **Grid** следующим XAML-кодом.

```
<Grid.Background>
    <ImageBrush ImageSource="watermark.png"  />
</Grid.Background>

<Grid.ColumnDefinitions>
    <ColumnDefinition Width="230" />
    <ColumnDefinition />
</Grid.ColumnDefinitions>

<Grid.RowDefinitions>
    <RowDefinition/>
    <RowDefinition Height="Auto"/>
    <RowDefinition />
    <RowDefinition Height="Auto"/>
</Grid.RowDefinitions>

<!-- People list -->

<Label Grid.Column="1" Style="{StaticResource headerTextStyle}" >
    View Expense Report
</Label>

<Border Grid.Column="1" Grid.Row="1" Style="{StaticResource listHeaderStyle}">
    <Label Style="{StaticResource listHeaderTextStyle}">Names</Label>
</Border>
```

```
<ListBox Name="peopleListBox" Grid.Column="1" Grid.Row="2">
    <ListBoxItem>Mike</ListBoxItem>
    <ListBoxItem>Lisa</ListBoxItem>
    <ListBoxItem>John</ListBoxItem>
    <ListBoxItem>Mary</ListBoxItem>
</ListBox>

<!-- View report button -->
<Button Grid.Column="1" Grid.Row="3" Click="Button_Click" Style="{StaticResource
  buttonStyle}">View</Button>
```

Свойства, определяющие внешний вид элементов управления (такие как **VerticalAlignment** и **FontFamily**), удаляются, и вместо них применяются стили. Например, стиль headerTextStyle применяется к тексту "View Expense Report" (элемент **Label**).

5. Откройте файл ExpenseReportPage.xaml.
6. Замените все содержимое между элементами **Grid** следующим XAML-кодом.

```
<Grid.Background>
    <ImageBrush ImageSource="watermark.png" />
</Grid.Background>
<Grid.ColumnDefinitions>
    <ColumnDefinition Width="230" />
    <ColumnDefinition />
</Grid.ColumnDefinitions>
<Grid.RowDefinitions>
    <RowDefinition Height="Auto" />
    <RowDefinition />
</Grid.RowDefinitions>


<Label Grid.Column="1" Style="{StaticResource headerTextStyle}">
    Expense Report For:
</Label>
<Grid Margin="10" Grid.Column="1" Grid.Row="1">

    <Grid.ColumnDefinitions>
        <ColumnDefinition />
        <ColumnDefinition />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition />
    </Grid.RowDefinitions>

    <!-- Name -->
```

```
    <StackPanel Grid.Column="0" Grid.ColumnSpan="2" Grid.Row="0"
  Orientation="Horizontal">
        <Label Style="{StaticResource labelStyle}">Name:</Label>
        <Label Style="{StaticResource labelStyle}"></Label>
    </StackPanel>

    <!-- Department -->
    <StackPanel Grid.Column="0" Grid.ColumnSpan="2" Grid.Row="1"
Orientation="Horizontal">
        <Label Style="{StaticResource labelStyle}">Department:</Label>
        <Label Style="{StaticResource labelStyle}"></Label>
    </StackPanel>

    <Grid Grid.Column="0" Grid.ColumnSpan="2" Grid.Row="2"
  VerticalAlignment="Top"
          HorizontalAlignment="Left">
        <!-- Expense type and Amount table -->
        <DataGrid ColumnHeaderStyle="{StaticResource columnHeaderStyle}"
                  AutoGenerateColumns="False" RowHeaderWidth="0" >
            <DataGrid.Columns>
                <DataGridTextColumn Header="ExpenseType" />
                <DataGridTextColumn Header="Amount"  />
            </DataGrid.Columns>
        </DataGrid>
    </Grid>
</Grid>
```

Таким образом, в элементы **Label** и **Border** будут добавлены стили.

7. Постройте и запустите приложение.

После добавления XAML-кода в этом упражнении приложение выглядит так же, как и перед обновлением с использованием стилей.

Упражнение 5. Привязка данных к элементу управления

Создание ресурса

В этом упражнении создаются XML-данные, привязанные к различным элементам управления. В качестве источника данных будет использоваться XML-файл, а для подключения к нему класс **XmlDataProvider**. Для организации связи с источником данных **XmlDataProvider** будет использовать свойства: Source – для установки источника данных (файла) и XPath – для задания пути внутри документа xml к целевому набору данных.

1. Добавьте в проект папку с именем Data.
2. В папку Data добавьте xml-файл – для этого воспользуйтесь командой контекстного меню папки: Добавить -> Создать элемент.
3. Вставьте следующий код, описывающий исходные данные:

```
<?xml version="1.0" encoding="utf-8" ?>
```

```
<Expenses>
  <Persons>
    <Person Name="Mike" Department="Legal">
        <Expense ExpenseType="Lunch" ExpenseAmount="50" />
        <Expense ExpenseType="Transportation" ExpenseAmount="50" />
    </Person>
    <Person Name="Lisa" Department="Marketing">
        <Expense ExpenseType="Document printing" ExpenseAmount="50"/>
        <Expense ExpenseType="Gift" ExpenseAmount="125" />
    </Person>
    <Person Name="John" Department="Engineering">
        <Expense ExpenseType="Magazine subscription" ExpenseAmount="50"/>
        <Expense ExpenseType="New machine" ExpenseAmount="600" />
        <Expense ExpenseType="Software" ExpenseAmount="500" />
    </Person>
    <Person Name="Mary" Department="Finance">
        <Expense ExpenseType="Dinner" ExpenseAmount="100" />
    </Person>
  </Persons>
   </Expenses>
```

4. Сохраните внесенные изменения.
5. Откройте файл ExpenseItHome.xaml.
6. В начале элемента **Grid**, добавьте следующий XAML-код для создания объекта **XmlDataProvider**, содержащего данные по каждому человеку.

Данные создаются в качестве ресурса объекта **Grid**. Обычно такие ресурсы загружаются в виде файла, как сделано в данном случае.

```
<Grid.Resources>
<!-- Expense Report Data -->
<XmlDataProvider x:Key="ExpenseDataSource" XPath Expenses/Persons"
  Source="Data/XMLData.xml">
</XmlDataProvider>

</Grid.Resources>
```

7. В ресурсе **Grid** добавьте следующий элемент **DataTemplate**, определяющий способ отображения данных в списке **ListBox**. По этому шаблону на каждый объект класса Person будет автоматически создаваться ListBoxItem в виде Label и его свойство Content будет привязано к Name:

```
<Grid.Resources>
...
<!-- Name item template -->
<DataTemplate x:Key="nameItemTemplate">
    <Label Content="{Binding XPath=@Name}"/>
</DataTemplate>
...
</Grid.Resources>
```

8. Замените существующий класс **ListBox** следующим XAML-кодом. Этот XAML-код обеспечивает привязку свойства **ItemsSource** элемента **ListBox** к источнику данных и применяет шаблон источника данных как элемент **ItemTemplate**.

```
<ListBox Name="peopleListBox" Grid.Column="1" Grid.Row="2"
         ItemsSource="{Binding Source={StaticResource ExpenseDataSource},
  XPath=Person}"
         ItemTemplate="{StaticResource nameItemTemplate}">
</ListBox>
```

Подключение данных к элементам управления

В этом разделе упражнения добавляется код, извлекающий текущий пункт, выбранный в списке персон на странице ExpenseItHome.xaml, и передающий его ссылку конструктору элемента ExpenseReportPage во время создания объекта. ExpenseReportPage задает контекст данных для переданного элемента, к которому будут привязаны элементы управления, определенные в файле ExpenseReportPage.xaml.

1. Откройте файл ExpenseReportPage.xaml.cs.
2. Добавьте конструктор, принимающий объект, чтобы можно было передать данные отчета о затратах выбранного человека.

```
public partial class ExpenseReportPage : Page
{
    public ExpenseReportPage()
    {
        InitializeComponent();
    }

    // Custom constructor to pass expense report data
    public ExpenseReportPage(object data):this()
    {
        // Bind to expense report data.
        this.DataContext = data;
    }
}
```

3. Откройте файл ExpenseItHome.xaml.cs.
4. Измените обработчик событий **Click** так, чтобы он вызывал новый конструктор и передавал ему данные отчета о затратах выбранного человека.

```
private void Button_Click(object sender, RoutedEventArgs e)
{
    // View Expense Report
    ExpenseReportPage expenseReportPage = new
  ExpenseReportPage(this.peopleListBox.SelectedItem);
    this.NavigationService.Navigate(expenseReportPage);
}
```

Стилизация данных с помощью шаблонов данных

В этом разделе обновляется UI каждого элемента в списках с привязкой данных с помощью шаблонов данных.

1. Откройте файл ExpenseReportPage.xaml.

2. Привяжите содержимое "Name" и "Department" элементов **Label** к соответствующему свойству источника данных:

```
<!-- Name -->
<StackPanel Grid.Column="0" Grid.ColumnSpan="2" Grid.Row="0"
  Orientation="Horizontal">
    <Label Style="{StaticResource labelStyle}">Name:</Label>
    <Label Style="{StaticResource labelStyle}" Content="{Binding
  XPath=@Name}"></Label>
</StackPanel>

<!-- Department -->
<StackPanel Grid.Column="0" Grid.ColumnSpan="2" Grid.Row="1"
  Orientation="Horizontal">
    <Label Style="{StaticResource labelStyle}">Department:</Label>
    <Label Style="{StaticResource labelStyle}" Content="{Binding
  XPath=@Department}"></Label>
</StackPanel>
```

3. После открывающего элемента **Grid** добавьте следующие шаблоны данных, определяющие способ отображения отчета о затратах:

```
<!--Templates to display expense report data-->
<Grid.Resources>
    <!-- Reason item template -->
    <DataTemplate x:Key="typeItemTemplate">
        <Label Content="{Binding XPath=@ExpenseType}"/>
    </DataTemplate>
    <!-- Amount item template -->
    <DataTemplate x:Key="amountItemTemplate">
        <Label Content="{Binding XPath=@ExpenseAmount}"/>
    </DataTemplate>
</Grid.Resources>
```

4. Примените шаблоны к столбцам **DataGrid**, в которых отображаются данные отчета о затратах. Замените существующий код в **DataGrid** на следующий:

```
<!-- Expense type and Amount table -->
<DataGrid ItemsSource="{Binding XPath=Expense}"
  ColumnHeaderStyle="{StaticResource columnHeaderStyle}"
  AutoGenerateColumns="False" RowHeaderWidth="0" >
    <DataGrid.Columns>
        <DataGridTextColumn Header="ExpenseType" Binding="{Binding
  XPath=@ExpenseType}"  />
        <DataGridTextColumn Header="Amount" Binding="{Binding
  XPath=@ExpenseAmount}" />
    </DataGrid.Columns>

</DataGrid>
```

5. Постройте и запустите приложение.

6. Выберите в списке имя человека и нажмите кнопку **View** для просмотра данных этого человека.

Лабораторная работа 2. Разработка приложения на основе шаблона MVVM

В этой лабораторной работе описывается реализация приложение WPF на основе паттерна MVVM (Model- View - ViewModel).

Данный паттерн содержит:

Model (модель) – представляет собой фундаментальные данные, необходимые для работы приложения (классы, структуры).

View (Вид/Представление) – определяет графический интерфейс.

ViewModel (Модель представления) – является с одной стороны абстракцией представления, а с другой предоставляет обертку данных из модели, которые подлежат связыванию. То есть она содержит модель, которая преобразована к представлению, а так же содержит в себе команды, которыми может пользоваться представление, чтобы влиять на модель. Таким образом, классы модели представления осуществляют связь между моделью и формой, отслеживают изменения в данных, произведенные пользователем и обрабатывают логику работы представления.

Упражнение 1. Создание инфраструктуры приложения MVVM

В этом упражнении создается инфраструктура приложения на основе паттерна MVVM, которая включает необходимые папки и настройки.

1. Создайте новый проект приложения WPF Visual C# с именем Wpf_MVVM.
2. Добавьте в проект три папки: **View**, **Model** и **ViewModel.**
3. В папке **View** должны находиться формы, поэтому переместите главную форму приложения (файл MainWindow.xaml) в эту папку.
4. Измените имя класса, чтобы из него было понятно к какой группе принадлежит класс: в файле MainWindow.xaml укажите

```
<Window x:Class="WpfApplication.View.MainWindowView"
```

5. Измените в файле MainWindow.xaml.cs пространство имен:

```
 namespace Wpf_MVVM.View
{
```

6. В файле App.xaml прописан путь к форме, которая запускается по умолчанию при старте программы. Так как форма перенесена и переименована, то путь необходимо изменить:

```
<Application x:Class="Wpf_MVVM.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             StartupUri="View/MainWindow.xaml">
```

7. Постройте и запустите приложение. Откроется стартовое окно.

Упражнение 2. Реализация элементов шаблона

В этом упражнении создается модель, реализующую сущность – человек, форма и настраивается связующее звено между этой формой и моделью – ViewModel.

1. Добавьте в папку **Model** модель – класс **PeopleModel**, в котором объявите два свойства имя и фамилия:

```
namespace Wpf_MVVM.Model
{
    class PeopleModel
    {
        #region Properties

        /// <summary>
        /// Get or set first name.
        /// </summary>
        public string FirstName { get; set; }

        /// <summary>
        /// Get or set last name.
        /// </summary>
        public string LastName { get; set; }

        #endregion
    }
}
```

2. Создайте связующее звено между формой и моделью. Для этого в папку ViewModel добавьте класс **MainViewModel**, в котором будет свойство типа нашей модели:

```
using Wpf_MVVM.Model;
using System.Windows;
using System.Windows.Input;

namespace Wpf_MVVM.ViewModel
{
    class MainViewModel
    {
        #region Constructor

        /// <summary>
        /// Constructor.
        /// </summary>
        public MainViewModel()
        {
            People = new PeopleModel
            {
```

```
                FirstName = "First name",
                LastName = "Last name"
            };
        }

        #endregion


        #region Properties

        /// <summary>
        /// Get or set people.
        /// </summary>
        public PeopleModel People { get; set; }

        #endregion
    }
}
```

3. Реализуйте интерфейс пользователя. Добавьте два текстовых блока **TextBlock** и четыре элемента **TextBox.** В файл MainWindow.xaml добавьте требуемый XAML-код:

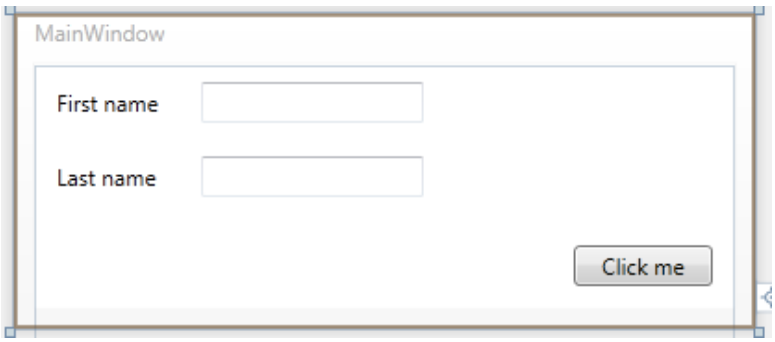
```
<Window x:Class="Wpf_MVVM.View.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="MainWindow" Height="170" Width="400">
    <Grid Height="169">
        <TextBlock HorizontalAlignment="Left" VerticalAlignment="Top"
  Margin="12,12,0,0"
                   Text="First name"/>
        <TextBlock HorizontalAlignment="Left" VerticalAlignment="Top"
  Margin="12,52,0,0"
                   Text="Last name"/>

        <TextBox HorizontalAlignment="Left" VerticalAlignment="Top"
  Margin="90,9,0,0" Width="120" />
        <TextBox HorizontalAlignment="Left" VerticalAlignment="Top"
  Margin="90,49,0,0" Width="120" />

        <Button Content="Click me" HorizontalAlignment="Left"
  Margin="291,97,0,0" VerticalAlignment="Top" Width="75" Command="{Binding
  ClickCommand}"/>

    </Grid>
</Window>
```

Вы должны получить следующий вид окна:



4. Укажите форме какой объект будет являться для нее ViewModel. Ссылка на этот объект должна находиться в свойстве **DataContext** формы. Поскольку **DataContext** у формы должен задаваться при создании формы удалите ссылку на нее в файле App.xaml:

```
StartupUri="View/MainWindow.xaml
```

5. В файле App.xaml.cs в конструкторе создайте форму, настройте ее свойство **DataContext** и непосредственно отобразите ее следующим образом:

```
 namespace Wpf_MVVM
{
    /// <summary>
    /// Логика взаимодействия для App.xaml
    /// </summary>
    public partial class App : Application
    {
        public App()
        {
            var mw = new MainWindow
            {
                DataContext = new MainViewModel()
            };

            mw.Show();
        }
    }
}
```

6. Привяжите элементы на форме к конкретным свойствам модели. Для этого опишите эти привязки в разметке кода MainWindow.xaml:

```
<TextBox HorizontalAlignment="Left" VerticalAlignment="Top" Margin="90,9,0,0" Wi
  dth="120" Text="{Binding Path=People.FirstName, Mode=TwoWay}"/>
<TextBox HorizontalAlignment="Left" VerticalAlignment="Top" Margin="90,49,0,0"
  Width="120" Text="{Binding Path=People.LastName, Mode=TwoWay}"/>
```

Для каждого элемента **TextBox** свойство **Text** привязано к свойству People.FirstName или People.LastName у ViewModel. Причем привязка указана как двусторонняя (**TwoWay**).

7. Постройте и запустите приложение. На форме отобразятся данные, которые были проинициализировали в конструкторе **MainViewModel**.

Упражнение 3. Привязка команды кнопки к ViewModel

В этом упражнении создается привязка команды кнопки формы к модели **MainViewModel**. Схематично логика работы следующая: **Кнопка → Команда → Метод.**

1. В класс **MainViewModel** добавьте свойство команды:

```
 #region Properties
/// <summary>
/// Get or set people.
/// </summary>
public PeopleModel People { get; set; }
#endregion
```

2. Реализуйте метод, который будет вызываться при срабатывании команды:

```
#region Methods

/// <summary>
/// Click method.
/// </summary>
private void ClickMethod()
{
    MessageBox.Show("Click command");
}

#endregion
```

И создайте экземпляр интерфейса **ICommand:**

```
#region Commands
 /// <summary>
/// Get or set ClickCommand.
/// </summary>
public ICommand ClickCommand { get; set; }
 #endregion
```

3. Для того чтобы привязать метод к команде создать класс (файл Command.cs в папке **ViewModel**), реализующий интерфейс ICommand:

```
using System.Windows.Input;

namespace Wpf_MVVM.ViewModel
{
    public class Command : ICommand
    {
        #region Constructor

        public Command(Action<object> action)
        {
            ExecuteDelegate = action;
        }
```

```
        #endregion

        #region Properties

        public Predicate<object> CanExecuteDelegate { get; set; }
        public Action<object> ExecuteDelegate { get; set; }

        #endregion

        #region ICommand Members

        public bool CanExecute(object parameter)
        {
            if (CanExecuteDelegate != null)
            {
                return CanExecuteDelegate(parameter);
            }

            return true;
        }

        public event EventHandler CanExecuteChanged
        {
            add { CommandManager.RequerySuggested += value; }
            remove { CommandManager.RequerySuggested -= value; }
        }

        public void Execute(object parameter)
        {
            if (ExecuteDelegate != null)
            {
                ExecuteDelegate(parameter);
            }
        }

        #endregion
    }
}
```

В классе реализованы методы **CanExecute()** – определяет метод, указывающий, может ли команда выполниться в текущем состоянии и **Execute()** – определяет метод, вызываемый, когда команда запускается.

Суть этого класса в следующем. Когда пользователь нажимает кнопку на форме, происходит попытка вызова метода **Execute()** интерфейса **ICommand**. Поэтому необходимо сделать

так, чтобы при вызове этого метода, внутри него происходил вызов нашего метода через делегат, который мы передаем в конструкторе класса.

При необходимости этот класс легко расширить и сделать вызов любого количества пользовательских методов при срабатывании команды.

4. В конструкторе класса **MainViewModel** привяжите метод к команде:

```
ClickCommand = new Command(arg => ClickMethod());
```

5. Укажите связь между командой и кнопкой в файле **MainWindow.xaml:**

```
<Button Content="Click me" HorizontalAlignment="Left" Margin="291,97,0,0"
  VerticalAlignment="Top" Width="75" Command="{Binding ClickCommand}"/>
```

6. Постройте и запустите приложение.

Упражнение 4. Настройка обновления свойств

В этом упражнении реализуется возможность автоматического обновления формы при изменении свойств. Модель, значения свойств которой должны автоматически обновляться на форме должна реализовывать интерфейс **INotifyPropertyChanged**.

Этот интерфейс описывает "механизм уведомления", всех привязанных к свойству "клиентов", о том, что значение свойства изменилось. Уведомление происходит через вызов метода **OnPropertyChanged()** реализация которого делает попытку вызова события **PropertyChanged** на которое подписываются все клиенты. Это событие и метод как раз описаны в интерфейсе **INotifyPropertyChanged**.

1. Откройте файл PeopleModel.cs.
2. Реализуйте для класса **PeopleModel** интерфейс **INotifyPropertyChanged:**

```
class PeopleModel : INotifyPropertyChanged
{ …
```

3. В классе **PeopleModel** определите событие **PropertyChanged** и метод **OnPropertyChanged(),** и реализуйте в нем попытку вызова события **PropertyChanged** на которое подписываются клиенты:

```
class PeopleModel : INotifyPropertyChanged
{
    #region Implement INotyfyPropertyChanged members

    public event PropertyChangedEventHandler PropertyChanged;

    protected virtual void OnPropertyChanged(string propertyName)
    {
        if (PropertyChanged != null)
        {
            PropertyChanged(this, new
PropertyChangedEventArgs(propertyName));
        }
    }
    #endregion
```

4. Измените свойства, так чтобы каждое свойство, при изменении его значения уведомляло всех клиентов о том, что его значение изменилось. Уведомление реализуйте через вызов метода **OnPropertyChanged()**:

```
#region Fields

private string _FirstName;
private string _LastName;

#endregion

#region Properties

/// <summary>
/// Get or set first name.
/// </summary>
public string FirstName
{
    get { return _FirstName; }
    set
    {
        if (_FirstName != value)
        {
            _FirstName = value;
            OnPropertyChanged("FirstName");
        }
    }
}


/// <summary>
/// Get or set last name.
/// </summary>
public string LastName
{
    get { return _LastName; }
    set
    {
        if (_LastName != value)
        {
            _LastName = value;
            OnPropertyChanged("LastName");
        }
    }
}
```

```
#endregion
```

5. Чтобы считать введенное значение в текстовое поле измените метод **ClickMethod()** в классе **MainViewModel**:

```
private void ClickMethod()
{
MessageBox.Show("Person - " + People.FirstName + " " + People.LastName);
}
```

6. Постройте и запустите приложение. Проверьте, что вводимая информация отображается в окне вывода.

Литература

1. Мак Дональд М. WPF: Windows Presentation Foundation в .NET 4.5 с примерами на C# 5.0 для профессионалов.: Пер. с англ. — М. : Вильямс, 2015. – 1024 с.
2. Натан А. WPF 4. Подробное руководство. - Пер. с англ. - СПб.: Символ-Плюс, 2011. – 880 с.
3. Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.0 на языке C#. 3-е изд.-СПб.: Питер, 2012. – 928 с.: ил.

Электронные ресурсы WPF:

4. https://docs.microsoft.com/ru-ru/dotnet/framework/wpf/index
5. http://www.wpftutorial.net/Home.html